# UNIT II

# 2 Compile and Build using MAVEN and GRADLE

## Syllabus

*Introduction, Installation of Maven, POM files, Maven Build lifecycle, Build phases(compile build, test, package) Maven Profiles, Maven repositories(local, central, global), Maven plugins, Maven create and build Artifacts, Dependency management, Installation of Gradle, Understand build using Gradle.*

## Contents

# Part I : Maven

## 2.1 Introduction

- Maven is a project management tool. Basically it is a comprehension tool.
- It is based on Project Object Model(POM).
- This tool is used for build, dependency and documentation.
- Maven simplifies and standardizes the project build process.
- Apache Maven is a popular build automation and project management tool used primarily for Java projects.
- The Maven can further be used in building and managing the projects written using languages like C#, ruby and other programming languages.

### Problems without Maven

If we choose not to use Maven for building the Java projects then we may encounter following problems -

1) **Dependency management :** Without Maven we need to manage all the project dependencies manually. It includes the complex tasks such as downloading library files, ensuring their compatibility with the undergoing project, managing the version control. It can be time-consuming and error-prone.
2) **Build automation :** In case of Struts, Hibernates, Spring framework we need to manually create Jar and War files. The dependencies of these Jar files need to be set up manually.
3) **Standardized project structure :** Maven creates a standardized project structure which can be easily grasped by the developers. Without Maven, it is difficult to set up and maintain the standard project structure that could be followed by all the team members.
4) **IDE integration :** IDE stands for Integrated Development Environment. There are many popular IDE such as Eclipse, IntelliJ IDEA, NetBeans and so on. Many popular IDE provide seamless integration with Maven, allowing us to import, build and manage Maven projects effortlessly. Without Maven, we need to configure the IDE manually.
5) **Lack of plugins and community support :** Maven has support for plugins that can simplify various tasks such as deployment and documentation generation. Without Maven, lack of Plugins forces the developer to perform deployment and documentation tasks manually. Which can be time consuming and error-prone.

### Features of Maven

Following features of Maven -

1) Maven provides **simple project setup** that follows best practices.
2) Maven provides **superior dependency management** including automatic updating, dependency closures.
3) It is extensible, with the **ability to easily write plugins** in Java or scripting languages.
4) Maven is able **to build any number of projects** into predefined output types such as a JAR, WAR, or distribution based on metadata about the project.
5) Maven is able to **work with multiple projects** at the same time, easily.
6) Maven encourages the use of a **central repository of** JARs and other dependencies. Maven comes with a mechanism that can be used to download any JARs required for building your project from a central JAR repository. This allows users of Maven to reuse JARs across projects and encourages communication between projects to ensure that backward compatibility issues are dealt with.

### Review Questions

1. *What is Maven ? What problems a developer can face if he/she does not use the project management tool like Maven ?*
2. *Enlist the features of Maven.*

## 2.2 Installation of Maven

Following steps are followed for installation of Maven

### Prerequisite :

Before installing the Maven, the Java must be installed on your system. The latest versions of Maven require JDK 8 or above installation.

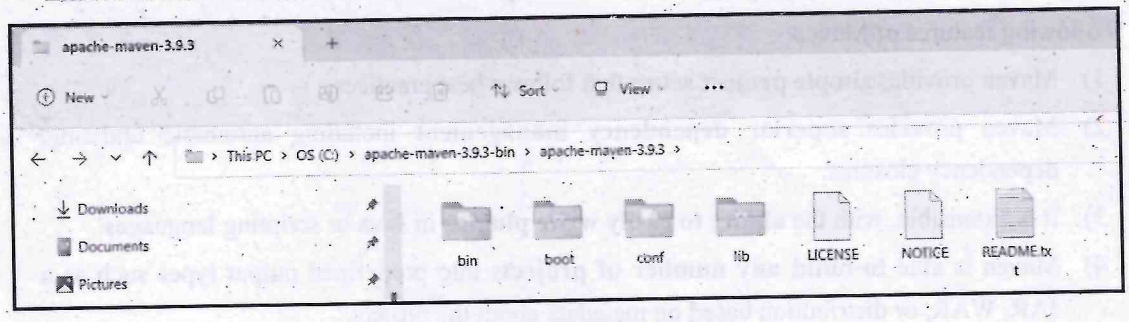
**Step 1 :** Download and extract Maven.

Go to the site **https://maven.apache.org/download.cgi.** Based on the operating system of your machine, choose the appropriate version.

For installing Maven on Windows we need Binary Zip files.

| | Link | Checksums | Signature |
|---|---|---|---|
| Binary tar.gz archive | apache-maven-3.9.3-bin.tar.gz | apache-maven-3.9.3-bin.tar.gz.sha512 | apache-maven-3.9.3-bin.tar.gz.asc |
| Binary zip archive | apache-maven-3.9.3-bin.zip | apache-maven-3.9.3-bin.zip.sha512 | apache-maven-3.9.3-bin.zip.asc |
| Source tar.gz archive | apache-maven-3.9.3-src.tar.gz | apache-maven-3.9.3-src.tar.gz.sha512 | apache-maven-3.9.3-src.tar.gz.asc |
| Source zip archive | apache-maven-3.9.3-src.zip | apache-maven-3.9.3-src.zip.sha512 | apache-maven-3.9.3-src.zip.asc |

Extract it. Now it will look like this :

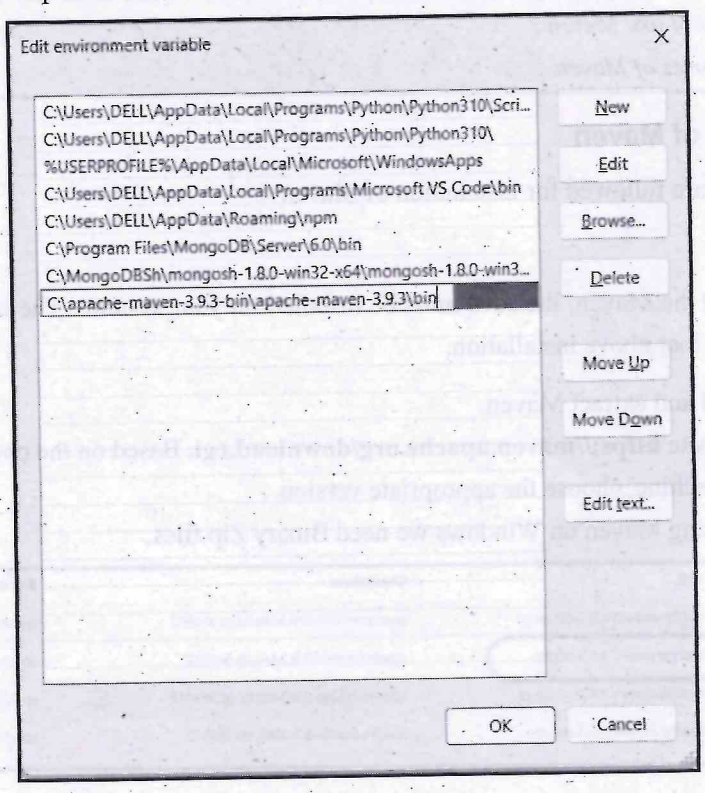



**Step 2 :** Set Environment variable with MAVEN_HOME variable.

Right click on **MyComputer**, click on **properties -> Advanced System Settings -> Environment variables -> click new button.**

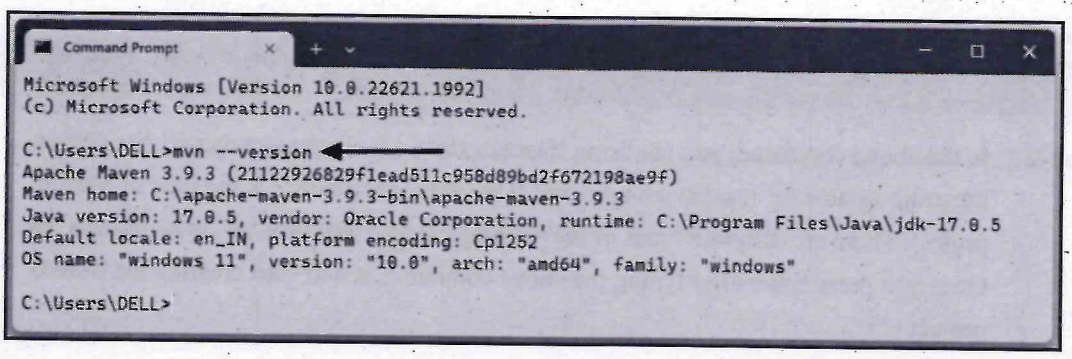
Now add MAVEN_HOME in variable name and path of maven in variable value. It must be the home directory of maven i.e. outer directory of bin. For example : C:\apache-maven-3.9.3-bin\apache-maven-3.9.3. It is displayed below :

Now, set the path for Maven. Just Right click on **MyComputer**, click on **properties -> Advanced System Settings -> Environment variables.** Click on **path** and click on New button. Then add the path C:\apache-maven-3.9.3-bin\apache-maven-3.9.3\bin at the end.



Click OK button.

**Step 3 :** Open command prompt window and issue the command **mvn - - version** at the command prompt. It prompts the Version of the Apache Maven, **Maven_Home** directory and Java Version. It is as shown below -




This shows that the Maven is installed successfully on your system.

## 2.3 Create and Build Maven Artifact

- In Maven terminology, an artifact is an output generated after a Maven project build. It can be, for example, a jar, war, or any other executable file.
- An artifact is an element that a project can either use or produce.
- Maven artifacts include five key elements, groupId, artifactId, version, packaging and classifier.
- The project's configuration file "pom.xml" describes how the artifact is build.
- Those are the elements we use to identify the artifact and are known as Maven coordinates.
  - **groupId :** It is sub element of project. It specifies the id for the project group.
  - **artifactId :** It is a sub element of project. This is generally refers to the name of the project. The artifact ID is also used as part of the name of the JAR, WAR or EAR file produced when building the project.
  - **version :** This is the sub element of project which specifies the version of the project.
  - **packaging :** It is used to define the packaging type such as jar, war etc.
- **Release Vs. Snapshot artifact**

  A **release artifact** indicates that the version is stable and can be used outside the development process. The outside development process can be testing, customer-qualification or preproduction.

  A **snapshot artifact** indicates that the project is under development. When we install or publish a component, Maven checks the version attribute. If it contains the string

"SNAPSHOT", Maven converts this key into a date and time value in UTC (Coordinated Universal Time) format.

**Creation of Maven Artifact using Command-line Argument**

**Step 1 :** Open the command prompt and issue the command -

```
$ mvn archetype:generate -DgroupId=com.mycompany.app -DartifactId=my-maven-app -
DarchetypeArtifactId=maven-archetype-quickstart -DarchetypeVersion=1.4 -DinteractiveMode=false
```

**Step 2 :** In the above command, you are using 'DartifactId' i.e. your project name(I have given the project name as 'my-maven-app' and 'DarchetypeArtifactId' is the type of Maven project. There are different types of maven projects like web projects, java projects, etc. Once you press Enter after typing the above command, it will start creating the Maven project

**Step 3 :** Following project structure will be created.

**my-maven-app**

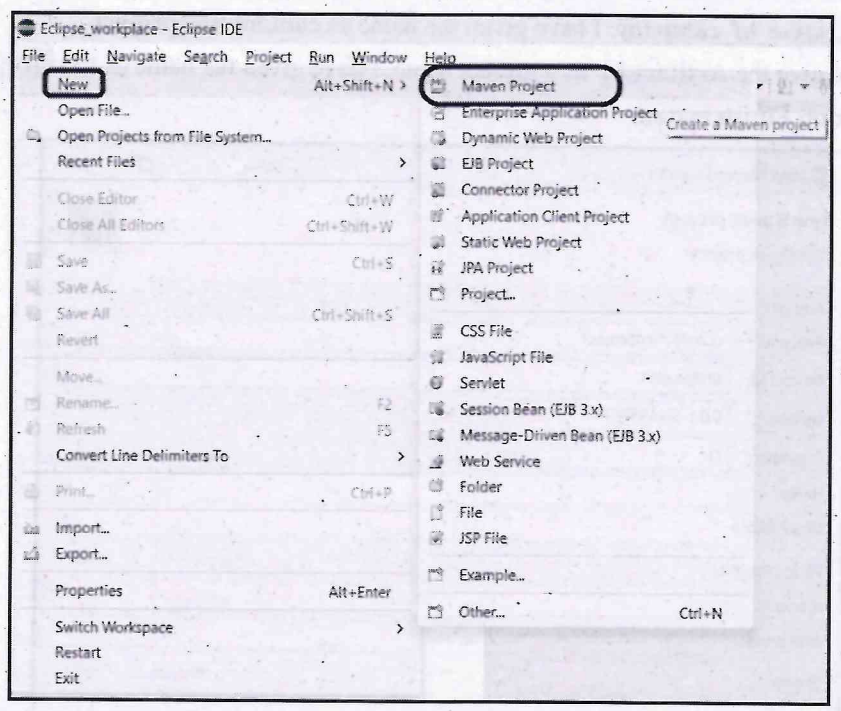
```
|-- pom.xml
`-- src
  |-- main
  |  `-- java
  |    `-- com
  |      `-- mycompany
  |        `-- app
  |          `-- App.java
  `-- test
    `-- java
      `-- com
        `-- mycompany
          `-- app
            `-- AppTest.java
```

Let us see a demo of creating an Artifact using Eclipse IDE.

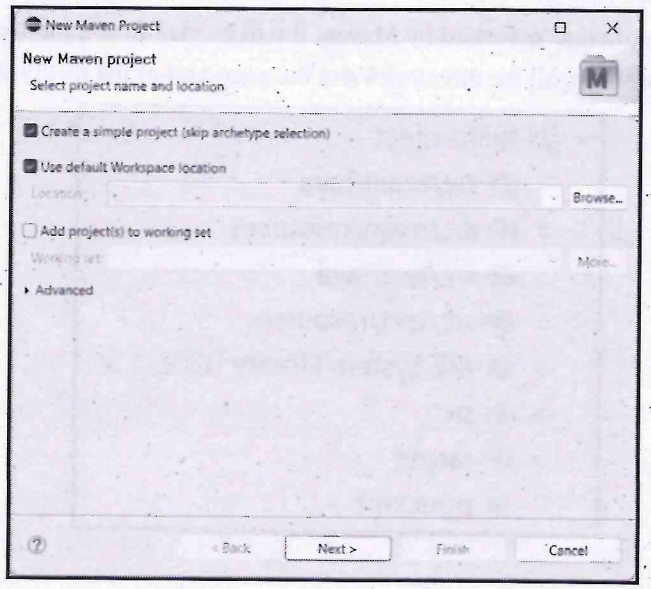
**Creation of Maven Project Using Eclipse IDE**

We can easily create a Maven project in Eclipse. Maven comes within Eclipse by default; there is no need to install it separately for Eclipse. Following are the steps to be followed for creating our first **Maven project** in **Eclipse**.

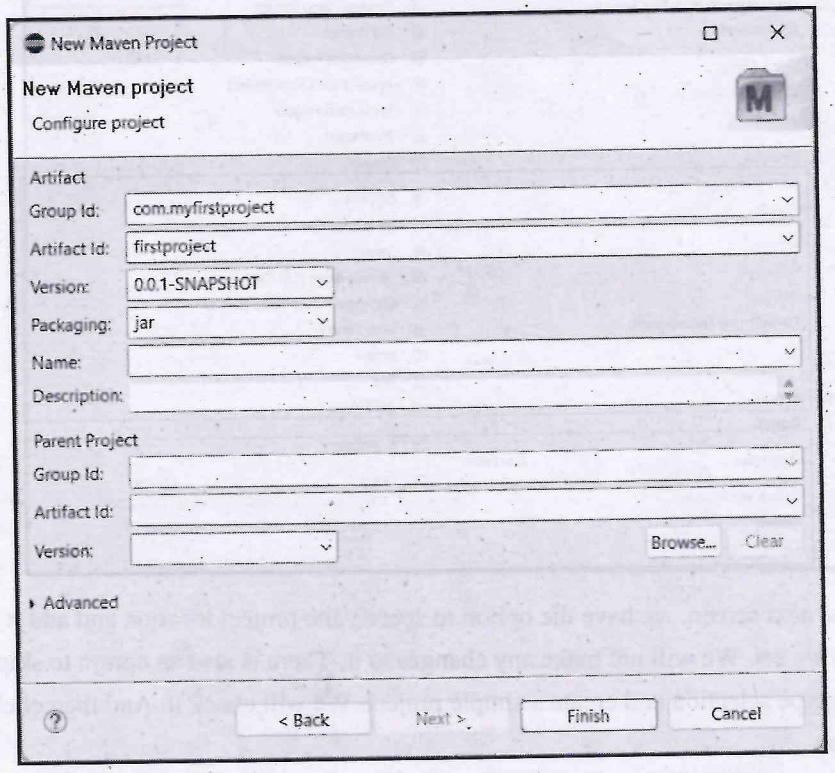
**Step 1 :** Click on **New->Maven Project**. Following screenshot illustrates it -



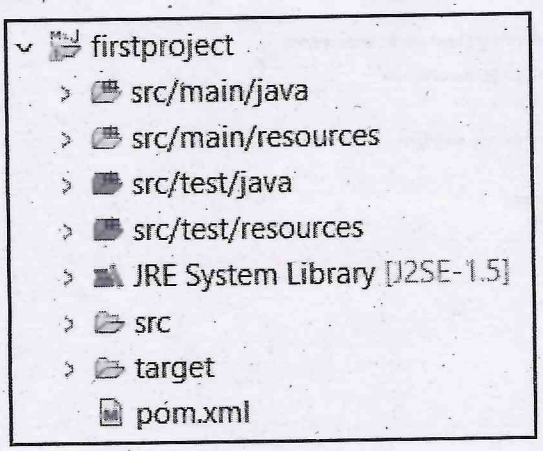

**Step 2 :** In the next screen, we have the option to specify the project location and add it to any working set. We will not make any changes to it. There is also an option to skip archetype selection and create a simple project. We will check it. And then click **Next** button.



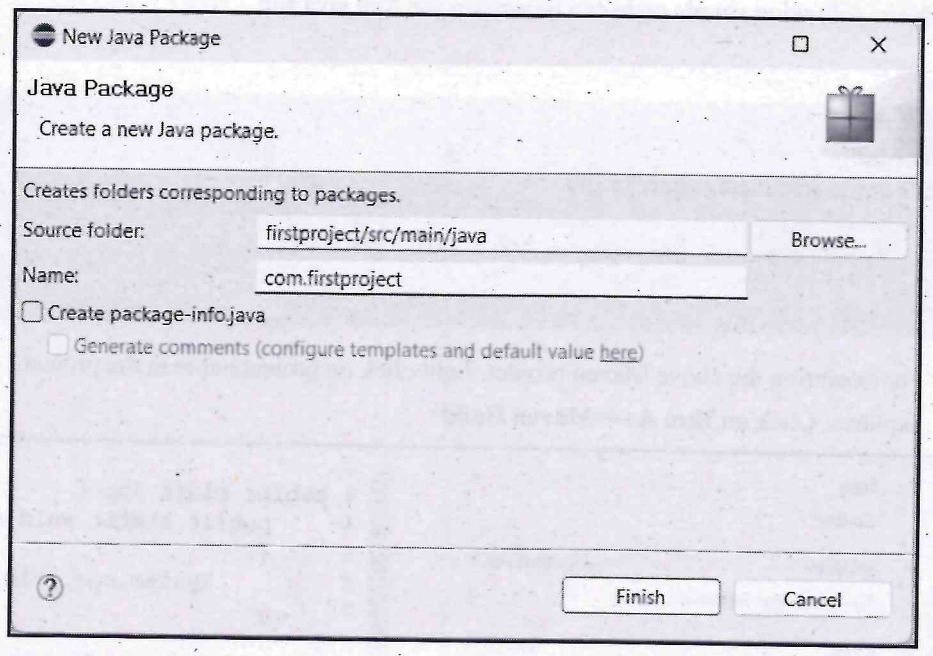
**Step 3 :** In the next screen we have to specify the **Group Id**. It will be given using ***com.name_of_company***. I have given the name as **com.myfirstproject**
Then give the **Artifact Id** as a project name. I have given the name as **firstproject**. Then click on **Finish** button.



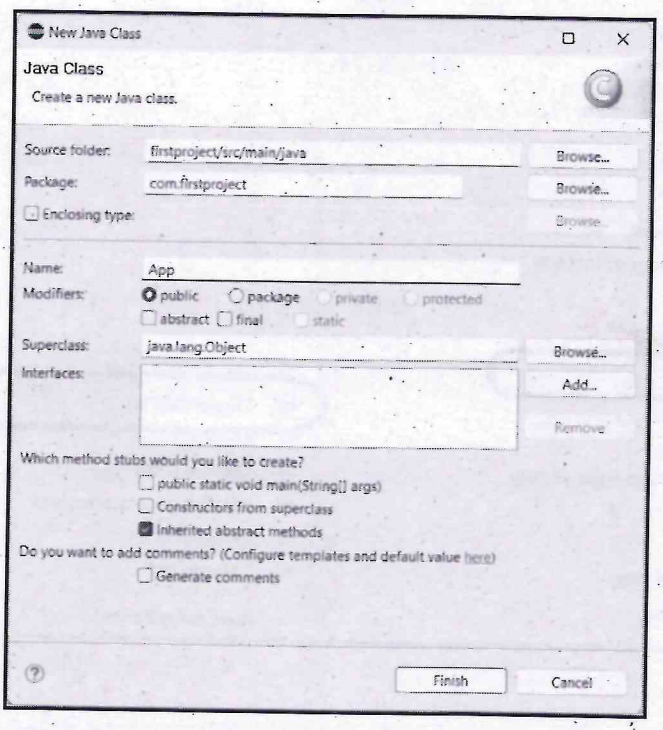

**Step 4 :** The project structure is created by Maven. It will be visible in the project explorer. The below image shows all the directories and the **pom.xml** of the newly created project.



**Step 5 :** Just right click on src/main/java folder and create a new package. I have given the name as **com.firstproject**. Following screenshot illustrates it -




Again right click on this package and create a **class** inside it. I have created a class by name **App**



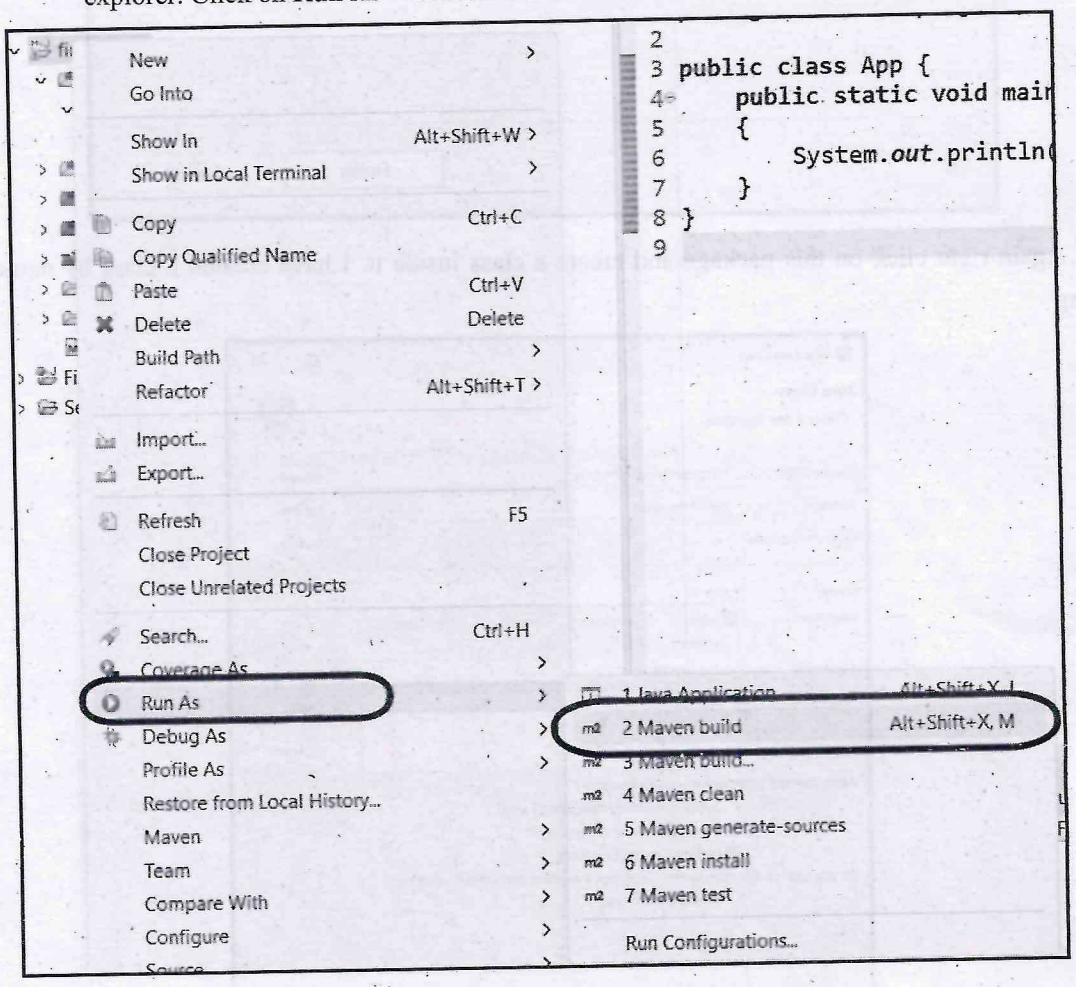
Click on **Finish** Button

**Step 6 :** Now, following simple code can be written for App.java file

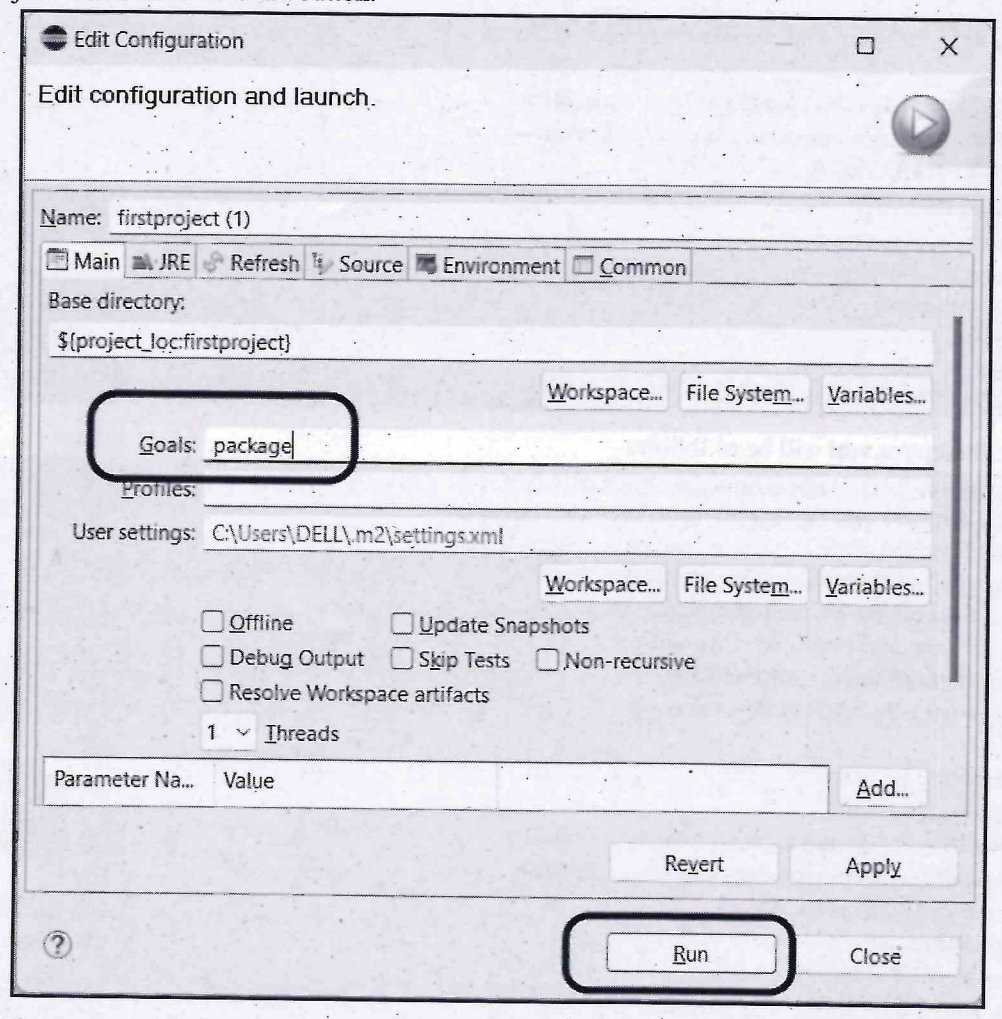
**App.java**

```
package com.firstproject;
public class App {
        public static void main(String[] args)
        {
                System.out.println("Welcome to First Maven Project!!!");
        }
}
```

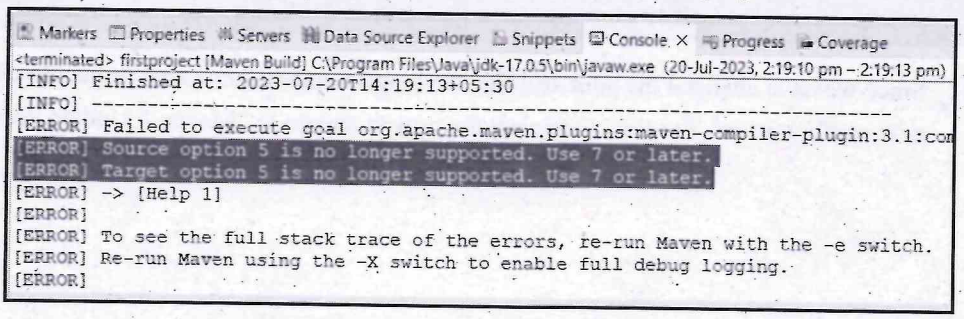
**Step 7 :** For executing the above Maven project, right click on project name in the project explorer. Click on **Run As-> Maven Build**



The "Edit Configuration" popup window will open. Enter the "Goals" as "package" to build the project and click on the Run button.




If you get the following error of **"Source option 5 is no longer supported. Use 7 or later"** then it is because of the auto-generated pom.xml file, which is not compatible with the latest Java versions. We can fix it by using the latest version of maven-compiler-plugin



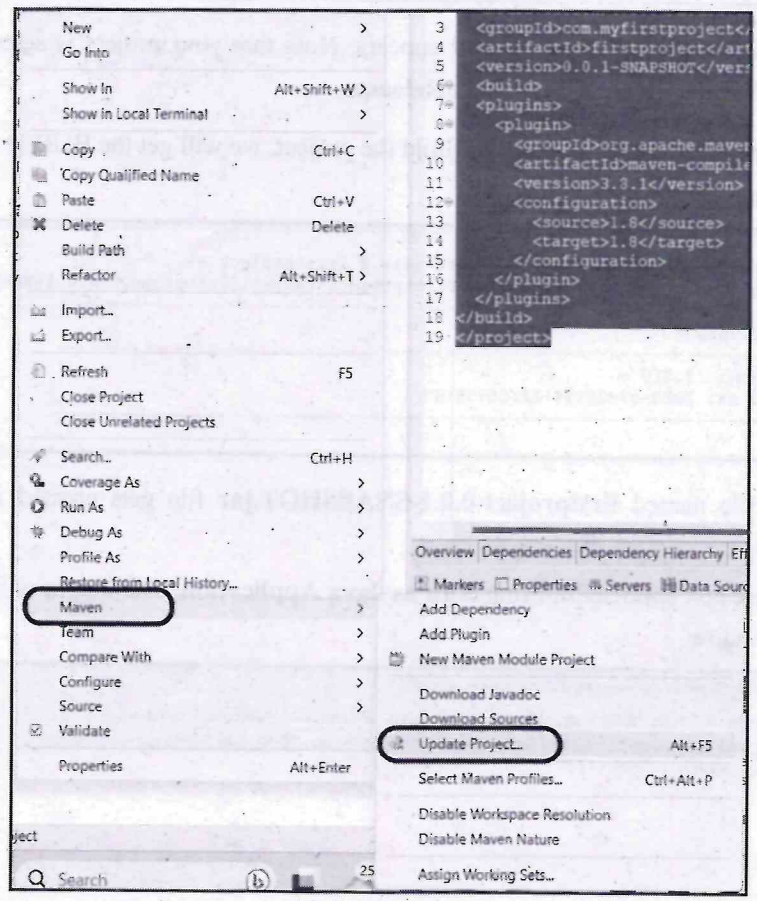
We will open out **pom.xml** file and edit it by adding the **maven-compiler-plugin**.

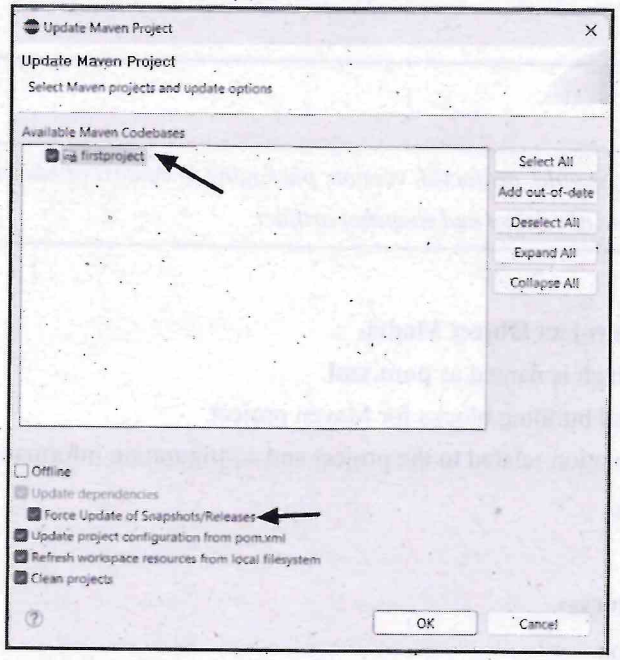
```
<build>
 <plugins>
  <plugin>
   <groupId>org.apache.maven.plugins</groupId>
   <artifactId>maven-compiler-plugin</artifactId>
   <version>3.8.1</version>
   <configuration>
    <source>1.8</source>
    <target>1.8</target>
   </configuration>
  </plugin>
 </plugins>
</build>
```

The edited **pom.xml** will be as follows -

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
 <modelVersion>4.0.0</modelVersion>
 <groupId>com.myfirstproject</groupId>
 <artifactId>firstproject</artifactId>
 <version>0.0.1-SNAPSHOT</version>
 <build>
 <plugins>
  <plugin>
   <groupId>org.apache.maven.plugins</groupId>
   <artifactId>maven-compiler-plugin</artifactId>
   <version>3.8.1</version>
   <configuration>
    <source>1.8</source>
    <target>1.8</target>
   </configuration>
  </plugin>
 </plugins>
</build>
</project>
```

**Step 8 :** Since we have changed the pom.xml file, we have to update the maven project to use the new configurations. For that, we Select the project and go to "Maven > Update Project".


New
Go Into
Show In Alt+Shift+W
Show in Local Terminal
Copy Ctrl+C
Copy Qualified Name
Paste Ctrl+V
Delete Delete
Build Path
Refactor Alt+Shift+T
Import...
Export...
Refresh F5
Close Project
Close Unrelated Projects
Search... Ctrl+H
Coverage As
Run As
Debug As
Profile As
Restore from Local History...
Maven
Team
Compare With
Configure
Source
Validate
Properties Alt+Enter
Add Dependency
Add Plugin
New Maven Module Project
Download Javadoc
Download Sources
Update Project... Alt+F5
Select Maven Profiles... Ctrl+Alt+P
Disable Workspace Resolution
Disable Maven Nature
Assign Working Sets...
Overview Dependencies Dependency Hierarchy
Markers Properties Servers
<groupId>com.myfirstproject
<artifactId>firstproject
<version>0.0.1-SNAPSHOT
<build>
<plugins>
<plugin>
<groupId>org.apache.maven
<artifactId>maven-compile
<version>3.3.1</version>
<configuration>
<source>1.8</source>
<target>1.8</target>
</configuration>
</plugin>
</plugins>
</build>
</project>


Update Maven Project
Update Maven Project
Select Maven projects and update options
Available Maven Codebases
firstproject
Select All
Add out-of-date
Deselect All
Expand All
Collapse All
Offline
Update dependencies
Force Update of Snapshots/Releases
Update project configuration from pom.xml
Refresh workspace resources from local filesystem
Clean projects
OK
Cancel

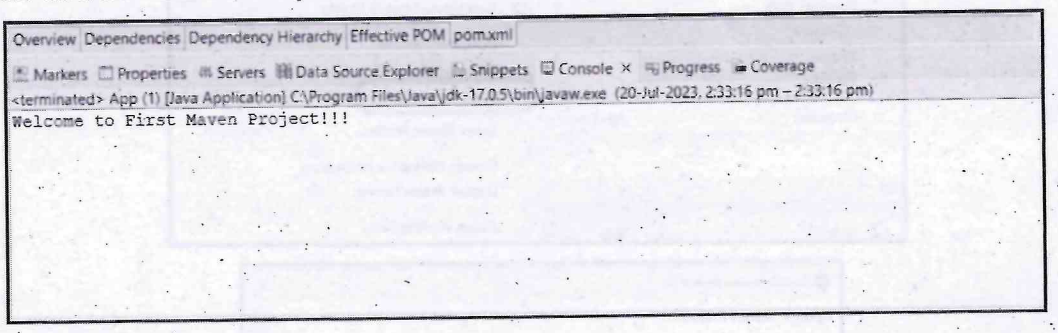
The Update Maven Project Window will appears. Note that your project is selected properly and check the **Enforce Update of Snapshots/Releases**

**Step 9 :** Now we have fixed pom.xml file. Build the project, we will get the BUILD SUCCESS message.

```
[INFO]
[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ firstproject ---
[INFO] Building jar: E:\Eclipse_workplace\firstproject\target\firstproject-0.0.1-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.310 s
[INFO] Finished at: 2023-07-20T14:32:06+05:30
[INFO] ------------------------------------------------------------------------
```

Thus our jar file named **firstproject-0.0.1-SNAPSHOT.jar** file gets created in the current working directory.

Now run the created application using **Run as Java Application**. The output will be displayed on the Console window.




## Review Questions

1. *Explain the terms groupId, artifactId, version, packaging in context of Maven.*
2. *Explain the purpose of release and snapshot artifact.*

## 2.4 POM Files

- POM stands for **Project Object Model**.
- It is XML file which is named as **pom.xml**.
- It is a fundamental building blocks for Maven project.
- It contains information related to the project and configuration information such as
  - **dependencies,**
  - **plugins,**
  - **source directory,**
  - **goals** and so on.

- Maven reads **pom.xml file** to accomplish its configuration and operations.
- The sample pom.xml is as follows -

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
 <modelVersion>4.0.0</modelVersion>
 <groupId>com.myfirstproject</groupId>
 <artifactId>firstproject</artifactId>
 <version>0.0.1-SNAPSHOT</version>
 <build>
 <plugins>
  <plugin>
   <groupId>org.apache.maven.plugins</groupId>
   <artifactId>maven-compiler-plugin</artifactId>
   <version>3.8.1</version>
   <configuration>
    <source>1.8</source>
    <target>1.8</target>
   </configuration>
  </plugin>
 </plugins>
</build>
</project>
```

**Elements of pom.xml file**

- **project :** It is a root element of the pom.xml file. This is the top level element.
- **modelVersion :** It Indicates the version model in which the current pom.xml is using. It is the sub element of project. It should be set to 4.0.0
- **groupId :** It is sub element of project. It specifies the id for the project group.
- **artifactId :** It is a sub element of project. This is generally refers to the name of the project. The artifact ID is also used as part of the name of the JAR, WAR or EAR file produced when building the project.
- **version :** This is the sub element of project which specifies the version of the project.

**Additional elements :**

- **packageing :** It is used to define the packaging type such as jar, war etc.
- **name :** It is used to define the name of the maven project.
- **url :** It is used to define the url of the project.

- **dependencies :** The POM lists all the external libraries and dependencies that the project relies on during the build process and runtime. Maven uses this information to automatically download the required dependencies from remote repositories.
- **dependency :** It defines a dependency. It is used inside dependencies.
- **Plugins :** Maven allows developers to use various plugins to **extend its functionality** during the build process. The POM defines which plugins are applied to the project and their respective **configurations**.
- **scope :** It is used to define the scope for this maven project. It can be compile, provided, runtime, test and system.

**Review Questions**

1. *Explain the purpose of Pom.xml file.*
2. *What are the elements of pom.xml file? Explain*

## 2.5 Maven Build Lifecycle

Maven build lifecycle is a sequence of steps that need to be followed in order to build a project.

Following are the phases in Maven build lifecycle -

1) **Validate :** It confirms that all the data necessary for the build is available.
2) **Compile :** In this phase the source code is compiled.
3) **Test Compile :** In this phase, it tests the compiled source code and copies into the destination directory.
4) **Test :** This phase tests code using a suitable unit testing method.
5) **Package :** In this phase, all the compiled files are gathered and packaged in a specified format.
6) **Install :** This phase installs the packages into the local repository.
7) **Deploy :** In this phase, the final package is deployed into a remote repository.

(Refer Fig. 2.5.1 on next page)

### Maven Goals

In Maven, each phase is a sequence of goals. The purpose of each goal in Maven is to perform in specific task. When we build our Maven project, we need to specify the **Goal**(Please, refer step 7 of section 2.3. For executing our Maven project we have specified the Goal as "package").

Some of the popular goals are -

- validate
- compile

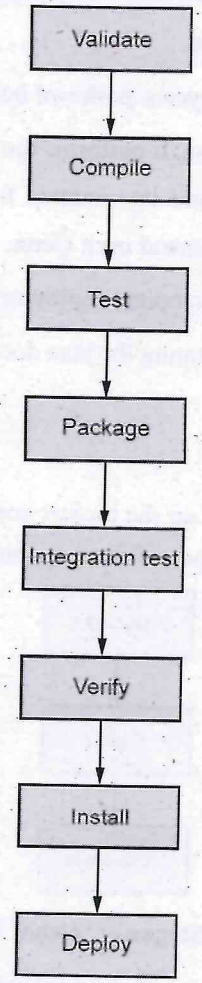



**Fig. 2.5.1 Maven build life cycle**

- test
- package
- install
- deploy

The build phases are executed sequentially. Any maven build phases that come before the specified phase is also executed. For example, if we run **mvn package** then it will execute compile, test and package phases of the project.

## Review Questions

1. *Explain the Maven build life cycle in detail*
2. *Explain the concet of Maven goals*

## 2.6 Built-in Build Life Cycle Phases

Maven comes with 3 built-in build life cycles as shown below :

1) **Clean :** In the Clean lifecycle phase, it performs the cleaning operation in which it deletes the build directory name target and its contents for a fresh build and deployment. To perform this operation we use command **mvn clean**.
2) **Default :** This phase handles the complete deployment of the project
3) **Site :** This phase handles the generating the java documentation of the project.

Let us discuss them in detail

### 1) Clean phase

The purpose of clean phase is to clean up the project and make it ready for fresh compile and deployment. Clean lifecycle performs 3 types of clean operations -

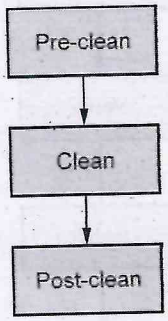



**Fig. 2.6.1 Stages in "clean" lifecycle**

- **Pre-clean :** This phase executes processes needed prior to the actual project cleaning.
- **Clean :** It removes all files generated by the previous build.
- **Post-clean :** It executes processes needed to finalize the project cleaning.

### 2) Default phase

- Maven default lifecycle handles everything related to compiling and packaging of the project. Below is the list of phases in the build lifecycle (default) of maven. These phases will be invoked through the maven commands.

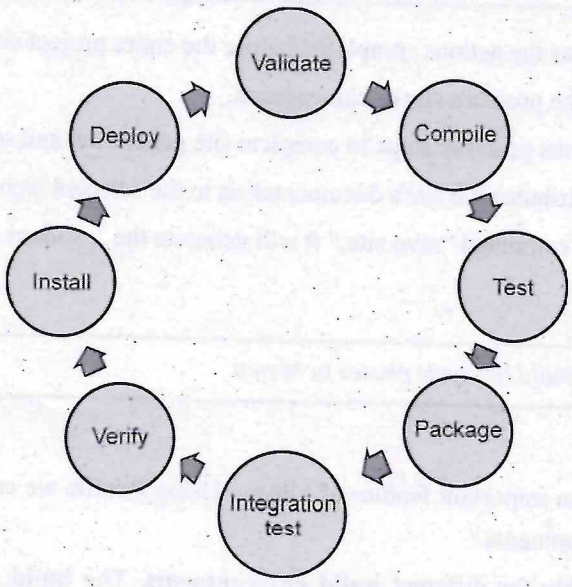



**Fig. 2.6.2 Phases in "Default" lifecycle**

| Lifecycle phase | Description |
|---|---|
| validate | Validates and ensures that the project is fine and perfect considering all the required information is made available for the build |
| compile | Compilation of the project source code. |
| test | Executing the tests using some suitable test framework. Note : these test cases are not considered for packaging and deploying |
| package | Packaging the successfully compiled and tested code to some distributable format - JAR, WAR, EAR |
| integration-test | Deploy the application to an environment where integration tests can be run. |
| verify | Performs action for a quality check and ensures the required criteria being met |
| install | Installing the application in the local repository. Any other project can use this as a dependency. |
| deploy | The final package will be copied to a remote repository, may be as a formal release. |

### 3) Site phase

This phase performs a crucial task of Maven which is detailed documentation of the project and preparing reports about project.

For Project documentation and site generation, in maven, we have a specific lifecycle dedicated for that, which has four phases and that are :

- **Pre-Site :** It executes the actions completed before the entire project site generation.
- **Site :** It generates the project's site documentation.
- **Post-site :** It performs post-site steps to complete site generation and set up site deployment.
- **Site deploy :** It distributes the site's documentation to the selected web server.

When we execute this command "mvn site," it will generate the Javadocs for the given Project.

**Review Question**

1. *Explain the built-in build life cycle phases in Maven*

## 2.7 Maven Profiles

- Maven Profile is an important feature of Maven. Using **Profile** we can customize the **build** for different environments.
- Profiles are portable for different **build environments**. The **build environment** means a specific environment for production, testing or development environment instances. So in order to configure these instances Maven provides the various features for the **build** using profile.
- The profiles are specified in a pom.xml file. Hence using a single pom.xml file, it possible to work in different environments such as production or development with the help of **profiles**.
- By defining the profiles it is possible to modify the pom.xml during the build time. That means we can configure separate environments for development and production instances. So, based on the parameters passed, the corresponding profiles are activated accordingly.

### 2.7.1 Different Types of Build Profiles

There are three types of build profiles -

1) **Per project :** It is defined in **pom.xml**.
2) **Per User/Developer :** It is defined in Maven **settings.xml (%USER_HOME%/.m2/settings.xml)**
3) **Global :** It is defined in Maven **global settings.xml (%M2_HOME%/conf/settings.xml)**

### 2.7.2 Elements of Profile

- The **profiles** element is in the **pom.xml**, it contains one or more **profile** elements. Since profiles override the default settings in a pom.xml, the profiles element is usually listed as the last element in a pom.xml.
- Each **profile** has to have an **id** element. This **id** element contains the name which is used to invoke this profile from the command-line. A profile is invoked by passing the

**-P<profile_id>** command-line argument to Maven.

- A profile element can contain many of the elements which can appear under the project element of a POM XML Document. In this example, we can overide the behavior of the **Compiler plugin** and we have to override the plugin configuration which is normally enclosed in a **build** and a **plugins** element.

```
<profiles>
    <profile>
        <id> </id>
        <build>
            <plugins>
                <plugin>
                    <groupId> </groupId>
                    <artifactId> </artifactId>
                    <version> </version>
                </plugin>
                <plugin>
                    <groupId> </groupId>
                    <artifactId> </artifactId>
                    <version> </version>
                </plugin>
            </plugins>
        </build>
    </profile>
</profiles>
```

## 2.7.3 Demo Example

We will create a profile demo example with the help of Eclipse. Start Eclipse IDE.

**Step 1 :** Click on **New->Maven Project**. Specify the **Group Id**. It will be given using **com.name_of_company**. I have given the name as **com.mycompany**
Then give the **Artifact Id** as a project name. I have given the name as **profiledemo**.
Then click on **Finish** button.

**Step 2 :** Create a package inside **src/main/java** folder of the project. I have named it as **com.profile**. Inside this package I have created a **App.java** file as follows -

**App.java**

```
package com.profiledemo;

public class App {
	public static void main(String[] args)
	{
		System.out.println("Simple Profile Demo Project");
	}
}
```

**Step 3 :** Edit the **pom.xml** file for adding the profiles for Test and Dev environment.

For **Test** environment

Id : TestEnv

Compiler Plugin Version : **3.8.1**

JAR Plugin Version : **3.0.0**

For **Dev** environment

Id : DevEnv

Compiler Plugin Version : **3.10.1**

JAR Plugin Version : **3.2.0**

The sample **pom.xml** file is as follows –

**pom.xml**

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
	<modelVersion>4.0.0</modelVersion>
	<groupId>com.mycompany</groupId>
	<artifactId>profiledemo</artifactId>
	<version>0.0.1-SNAPSHOT</version>
	<profiles>
		<profile>
			<id>TestEnv</id>
			<build>
			<plugins>
				<plugin>
					<groupId>org.apache.maven.plugins</groupId>
					<artifactId>maven-compiler-plugin</artifactId>
					<version>3.8.1</version>
					<configuration>
						<source>1.8</source>
						<target>1.8</target>
```

```
                    </configuration>
                    </plugin>
                    <plugin>
                        <groupId>org.apache.maven.plugins</groupId>
                        <artifactId>maven-jar-plugin</artifactId>
                    <version>3.0.0</version>
                </plugin>
                </plugins>
            </build>
        </profile>
        <profile>
            <id>DevEnv</id>
            <build>
            <plugins>
                <plugin>
                    <groupId>org.apache.maven.plugins</groupId>
                    <artifactId>maven-compiler-plugin</artifactId>
                    <version>3.10.1</version>
                    </plugin>
                    <plugin>
                        <groupId>org.apache.maven.plugins</groupId>
                        <artifactId>maven-jar-plugin</artifactId>
                    <version>3.2.0</version>
                </plugin>
                </plugins>
            </build>
        </profile>
    </profiles>
</project>
```

**Code explanation :** In above pom.xml file,

1) We have created two profiles using the tag <profile>
2) Each profile has some id. The id of first profile is **TestEnv** and the id of second profile is **DevEnv**.
3) We have used different versions of compiler plugin and jar-plugins.
4) We activate the profile by using the **mvn install -P <profile_id>**

Just refer the following output screenshot.

**Output 1**

If we execute the above project using the command

```
> mvn install -P TestEnv
```

We get following output

```
[INFO] Copying 0 resource
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ profiledemo ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding Cp1252, i.e. build is platform dependent!
[INFO]
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ profiledemo ---
[INFO]
[INFO] --- maven-jar-plugin:3.0.0:jar (default-jar) @ profiledemo ---
[INFO] Downloading from : https://repo.maven.apache.org/maven2/org/apache/maven/maven-archiver/3.0.2/maven-archiver-3.0.2.pom
[INFO] Downloaded from : https://repo.maven.apache.org/maven2/org/apache/maven/maven-archiver/3.0.2/maven-archiver-3.0.2.pom (4.1 kB at 3.6 k
[INFO] Downloading from : https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-components/22/maven-shared-components-22.
[INFO] Downloaded from : https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-components/22/maven-shared-components-22.p
```

Note that in above output, the maven-compiler-plugin version is 3.8.1 and maven-jar-plugin version is 3.0.0

**Output 2**

If we execute the above project using the command

```
> mvn install -P DevEnv
```

We get following output

```
[INFO]
[INFO] --- maven-compiler-plugin:3.10.1:testCompile (default-testCompile) @ profiledemo ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ profiledemo ---
[INFO]
[INFO] --- maven-jar-plugin:3.2.0:jar (default-jar) @ profiledemo ---
[INFO] Downloading from : https://repo.maven.apache.org/maven2/org/apache/maven/shared/file-manager
[INFO] Downloaded from : https://repo.maven.apache.org/maven2/org/apache/maven/shared/file-managem
[INFO] Downloading from : https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared
```

Note that in above output, the maven-compiler-plugin version is 3.10.1 and maven-jar-plugin version is 3.2.0

### Advantages of using profile

1) One of the major advantage of maven is creating profile based dependencies. Instead of creating separate POM.xml, we can create a profile and configure dependencies for that specific profile. This helps us to declare all the dependencies in one pom.xml.
2) Multiple dependencies can be executed without replicating the code. Within the same working project, multiple dependencies can be tested at a time.

### Review Questions

1. *What is Maven profile ? Explain it with the help of example*
2. *What are the types of build profiles?*
3. *Explain the elements of profile in Maven*
4. *What are the advantages of using Maven profile ?*

## 2.8 Maven Repositories

Maven repository is a directory where all the project jars, plugins, library jars or any other project related artifacts are stored and these can be accessed by Maven easily.

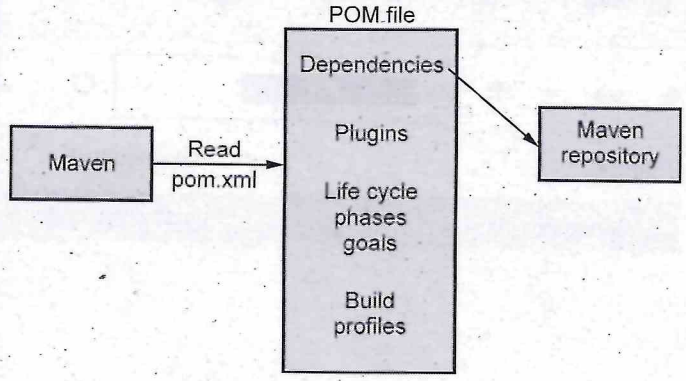



**Fig. 2.8.1 Use of Pom.xml file**

There are three types of repositories in Maven -

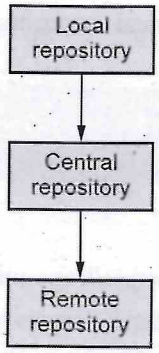
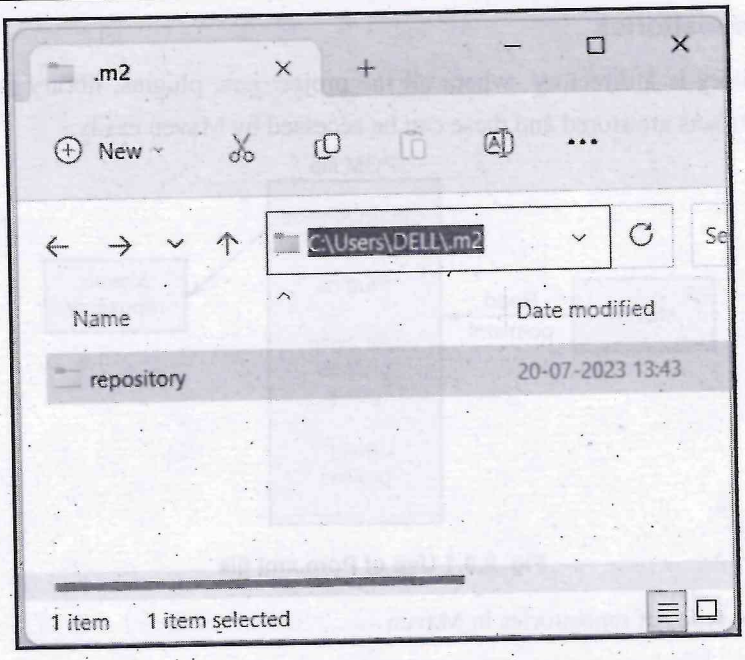



**Fig. 2.8.2 Types of repository**

### 1) Local repository :

Maven **local repository** is located in developer's machine. When we execute any Maven project that requires dependencies, Maven downloads these dependencies from remote servers and store them on developer's machine.

By default Maven creates local depository inside **user's** home directory i.e. C:/Users./m2 directory. Following Screenshot shows this.




We can change the location of local repository using the **settings.xml file** using **localRepository** tag.

```
<settings>
        <localRepository> C:\M2 </localRepositoy>
</settings>
```

**2) Central repository :**

Maven Central can be accessed from **https://repo.maven.apache.org/maven2/**. Whenever we execute our Maven project then, required dependencies are first searched in the local repository, if those dependencies are not present in the local repository then those are downloaded from the central repository and are stored in the local repository for the future use.

**3) Remote repository :**

Apart from central repository, we may need to deploy artifacts from remote locations. Such repositories are called as **remote repositories**. These Maven remote repository work exactly the same way as Maven's central repository. Whenever an artifact is needed from these repositories, it is first downloaded to the developer's local repository and then it is used.

**Review Question**

1. *Explain Maven repositories in detail.*

## 2.9 Maven Plugins

Maven Plugin is a central part of Maven framework which is used to perform specific goal. There are two types of Maven plugins -

1. **Build plugin :** These plugins are declared inside <build> element. These plugins are executed at the time of build.
2. **Reporting plugin :** These plugins are declared inside the element <reporting>. These plugins are executed at the time of site generation.

Following is a list of some **core plugins**

- **clean :** Clean up after the build.
- **compiler :** Compiles Java sources.
- **deploy :** Deploy the built artifact to the remote repository.
- **failsafe :** Run the JUnit integration tests in an isolated classloader.
- **install :** Install the built artifact into the local repository.
- **resources :** Copy the resources to the output directory for including in the JAR.
- **site :** Generate a site for the current project.
- **surefire :** Run the JUnit unit tests in an isolated classloader.
- **verifier :** Useful for integration tests -it verifies the existence of certain conditions.

**Review Question**

1. *What is Maven Plugin ? Explain any three core plugins used in Maven*

## 2.10 Dependency Management

- In Maven, a dependency is just another archive-JAR, ZIP, and so on-which our current project needs in order to compile, build, test, and/or run.
- These project dependencies are collectively specified in the pom.xml file, inside of a **<dependencies>** tag.
- When we run a maven build or execute a maven goal, these dependencies are resolved and then loaded from the local repository.
- If these dependencies are not present in the local repository, then Maven will download them from a remote repository and cache them in the local repository.
- Dependency management is an important activity because to build a product we need several library files, plugins or some external dependencies. We have to be sure that all these components are compatible for building our product. This task can be taken care by dependency management.

- **Example of dependency**

Here is a dependency section of **pom.xml** file.

```
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.12</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-core</artifactId>
    <version>4.3.5.RELEASE</version>
  </dependency>

</dependencies>
```

Inside the **<dependencies>** tag separate dependencies are specified by each **<dependency>** tag. Each dependency is supposed to have at least three main tags : groupId, artifactId.

- **groupId :** It is a sub element of the project tag, indicates a unique identifier of a group the project is created. Typically fully qualified domain name.
- **artifactId :** This element indicates a unique base name-primary artifact generated by a particular project.
- **Version :** This element specifies the version for the artifact under the given group.

### Concept of transitive dependency

Transitive dependency means when library A depends upon library B and library B depends upon library C then A depends upon both B and C.

In Maven using the command **dependency:tree** we can view all the transitive dependencies of our project. The command is

```
$ mvn dependency:tree
```

### Excluding dependencies

There can be version mismatch issues(sometimes due to transitive dependencies) between the project artifacts and artifacts from the platform of deployments, such as Tomcat or another server.

To resolve such version mismatch issues, maven provides <exclusion> tag, in order to break the transitive dependency.

**For example**

```
<project>
 ...
 <dependencies>
  <dependency>
   <groupId>myprojects.ProjectA</groupId>
   <artifactId>Project-A</artifactId>
   <version>1.0</version>
   <scope>compile</scope>
   <exclusions>
    <exclusion>  <!-- declare the exclusion here -->
     <groupId> myprojects.ProjectB</groupId>
     <artifactId>Project-B</artifactId>
    </exclusion>
   </exclusions>
  </dependency>
 </dependencies>
</project>
```

**Review Questions**

1. *Explain the Maven dependency management process in detail.*
2. *What is transitive dependency in Maven?*

## 2.11 Disadvantages of Maven

Following are some disadvantages or limitations of Maven -

1) Installation of Maven in each working system is required for executing the Maven based project.
2) We can not implement the dependency using Maven if Maven code for existing dependency is not available.
3) Maven is slow in execution of project.
4) It always need XML based file named **pom.xml** for configuration of the project.

**Review Question**

1. *Enlist the disadvantages of Maven.*

## Part II : Gradle

## 2.12 Introduction to Gradle

- Gradle is an Open source **build automation tool**.
- The build automation tool is a tool that automates the creation of software build. Build automation is the process of automating the retrieval of source code, compiling it into binary code, executing automated tests and publishing it into a shared, centralized repository.
- It is launched in 2007 but the stable release came in 2009. Finally, Gradle came into picture in 2012 by overcoming the drawbacks of both Ant and Maven.
- Gradle tools takes the advantages of Ant and Maven. It removes the disadvantages of Ant and Maven.
- Gradle is written in **Groovy language**.
- Gradle is a general purpose build tool and its main **focus is Java projects**.

### Features of Gradle

1) **Free and Open Source :** It is a open source tool.
2) **High Performance :** Gradle finishes the tasks efficiently by considering the outputs from previous execution.
3) **Better Support :** Gradle is popular for providing the support. The Gradle can provide support to build ANT projects. Tasks can be imported from ANT build projects and can be used in Gradle project.
4) **Scaling :** Gradle can increase the productivity, from simple and single project to huge multi-project builds.
5) **Multi-project builds :** Gradle supports the multi-project builds. It takes care of building all subprojects.
6) **IDE Support :** Gradle has support for several IDEs. Gradle also generates the required solution files to load a project into Visual Studio.

### Review Question

1. *What is Gradle ? Enlist its features.*

## 2.13 Installation of Gradle

Gradle is a Java based Tool. Hence the Java 8 or higher version must be installed before installing the Gradle.

If the Java is already installed or not is checked by issuing the command for Java version number at the command prompt

```
$java –version
```

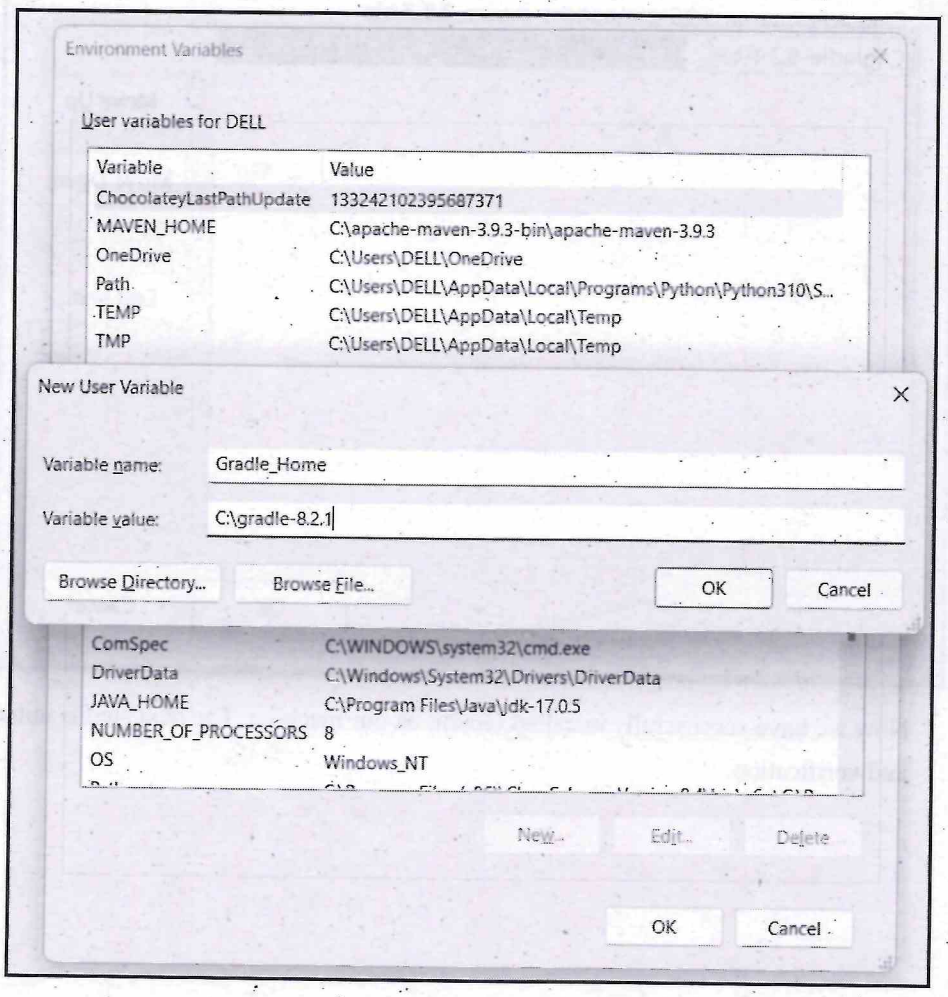
The version number is displayed if the Java is already installed on your machine.

**Step 1 :** Visit the official web page for Gradle https://gradle.org/install/

**Step 2 :** Select the **Binary-Only** option, and it will start downloading the .zip file on your machine.

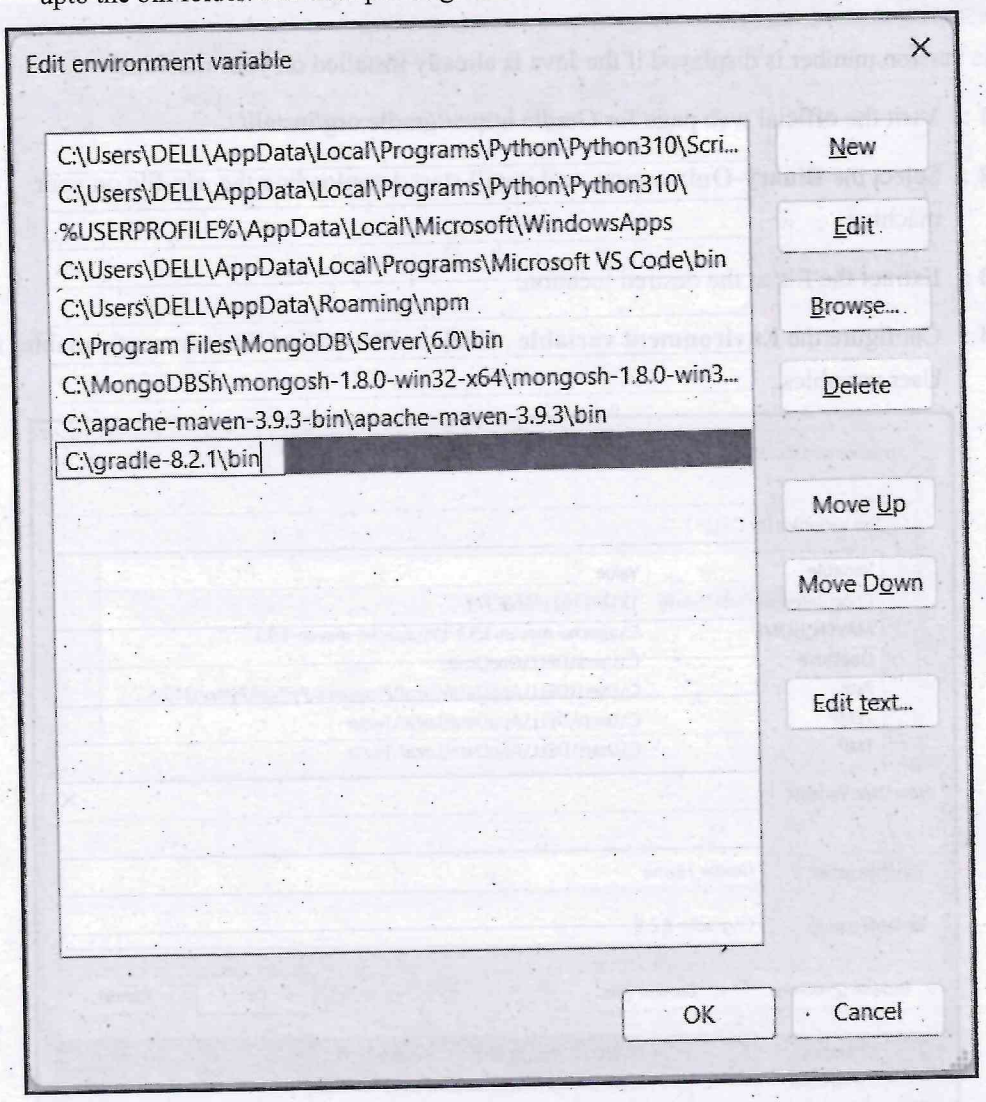
**Step 3 :** Extract the file at the desired location.

**Step 4 :** Configure the **Environment variable** . Click on **New** tab of Environment variables for User variables.



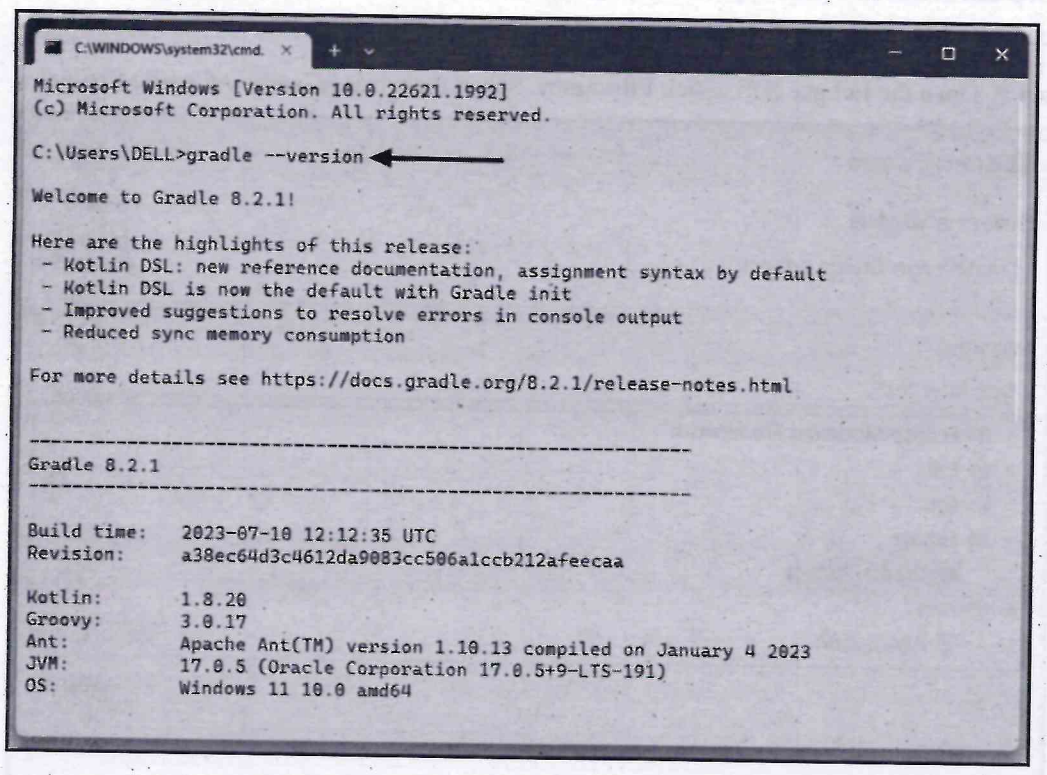

Click **Ok** button.

Then click on **path** and click on **Edit** button. Click on New button. Then add the path upto the bin folder. For example C:\gradle-8.2.1\bin. Then click **Ok**.




Now we have successfully installed Gradle on our machine. The next step is initialization and verification.

**Step 5 :** Open command prompt and issue the command **gradle --version**. It will display the version of gradle.




If we issue the command **gradle** at the command prompt then the gradle daemon will start.

## 2.14 Core Concepts

- **Project :** Gradle project represents the application that can be deployed to the staging environment.
- **Task :** A task refers to a piece of work performed by a build. For example - the task cane be creating a Jar file, compiling classes, or making JavaDoc
- **Build Script :** Gradle builds the build script for - **1. project 2. task**. Every Gradle build represents one or more projects. The build script is written using the domain specification language called **groovy**. This script is saved as **build.gradle**.

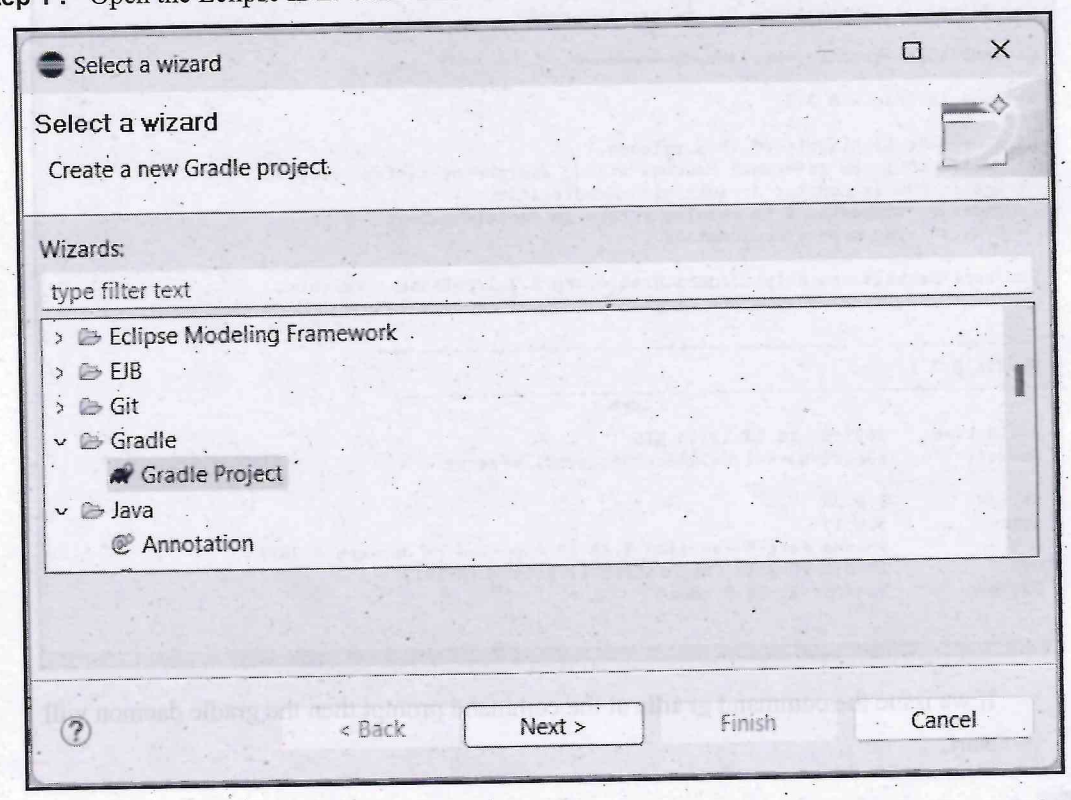
**Review Question**

1. *Explain the terms - project, task and build script in context of Gradle.*

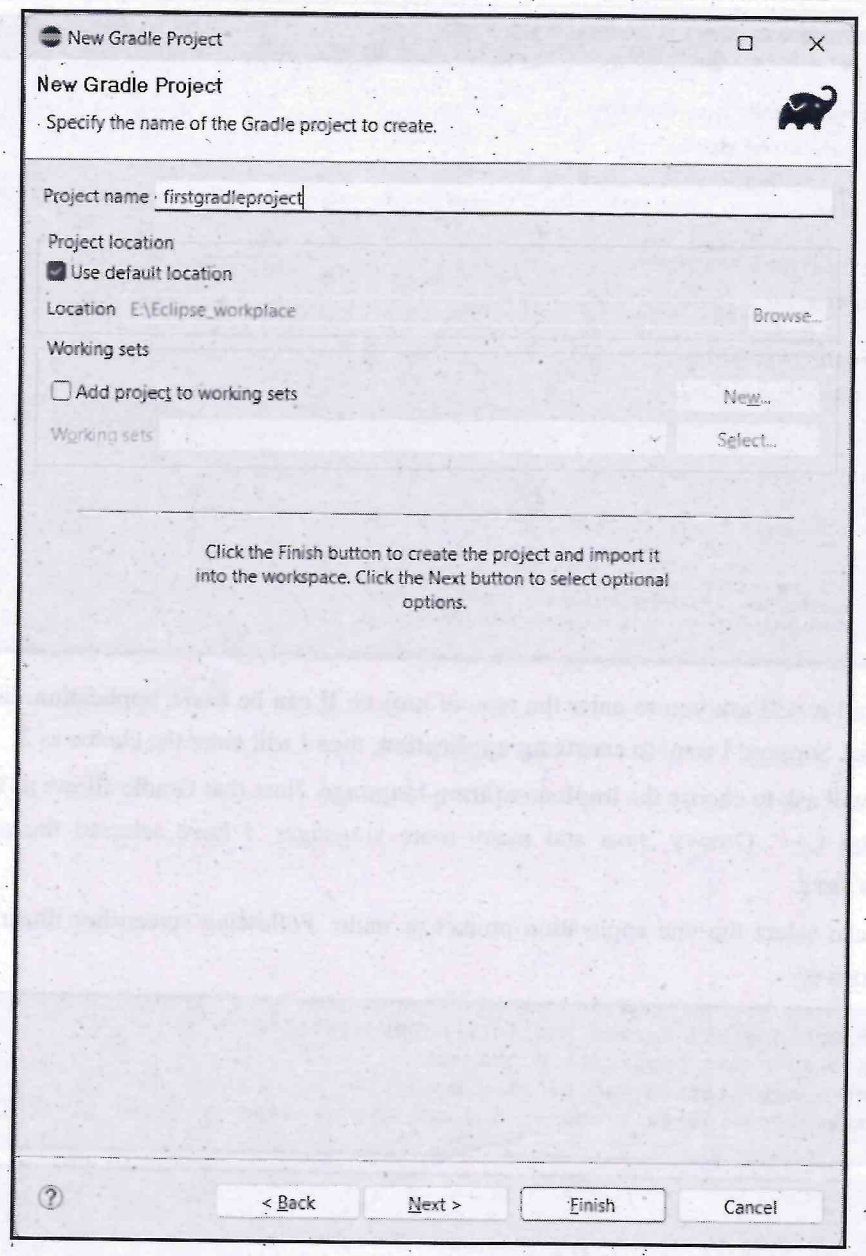
## 2.15 Building Basic Application using Gradle

We can create a Gradle application project either using some IDE or using the commands at the command-line prompt. First let us see how to create a Gradle application using Eclipse IDE.

**Step 1 :** Open the Eclipse IDE. Click **File->new**. Select the wizard **Gradle->Gradle Project** .




**Step 2 :** Click on Next button and give suitable project name. I have given the name **firstgradleproject**. Following screenshot illustrates it -



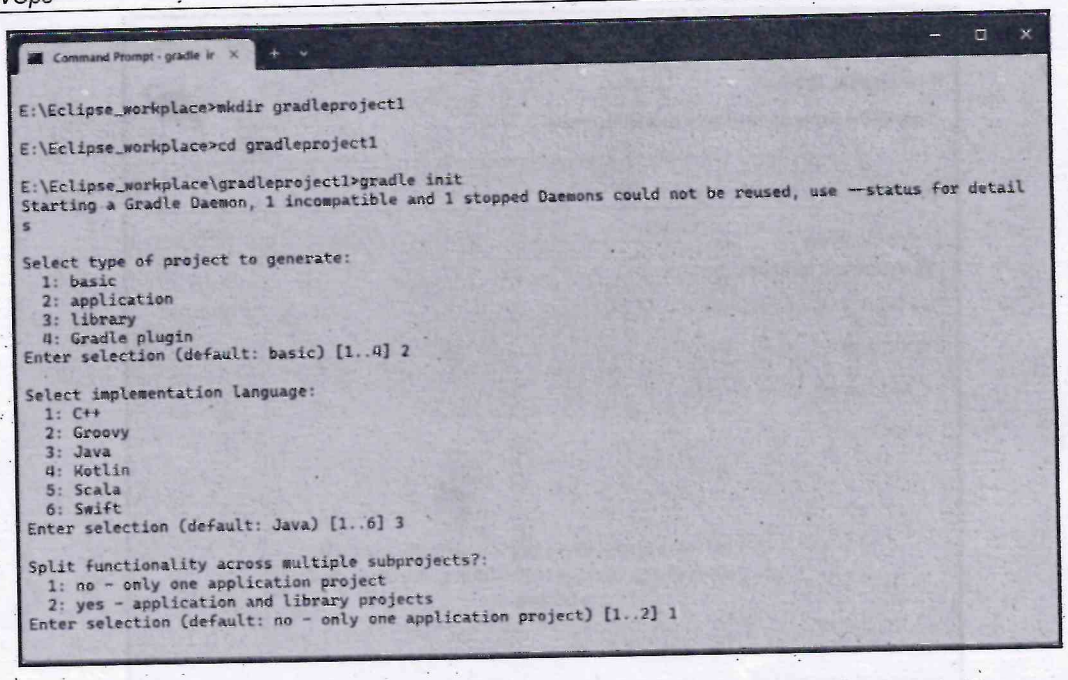

Finally click on **Finish** button.

### Creating Gradle project using Command-prompt

We can create a gradle project using the command-line by issuing the following command -

```
$ gradle init
```

Then it will ask you the desired configuration




First of all it will ask you to enter the type of project. It can be basic, application, library or a gradle plugin. Suppose I want to **create an application**, then I will enter the choice as 2.

Then it will ask to choose the **implementation language**. Note that Gradle allows us to create a project using C++, Groovy, Java and many more languages. I have selected the application language as **Java**.

Next I can select the one application project or multi. Following screenshot illustrates these configurations is

```
Split functionality across multiple subprojects?:
  1: no - only one application project
  2: yes - application and library projects
Enter selection (default: no - only one application project) [1..2] 1
```

```
Select build script DSL:
  1: Groovy
  2: Kotlin
Enter selection (default: Groovy) [1..2] 1

Generate build using new APIs and behavior (some features may change in the next minor release)? (default:
no) [yes, no]
o
```

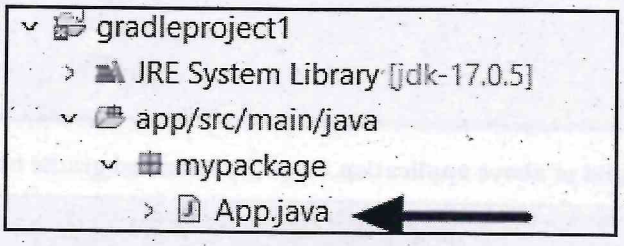
```
Select test framework:
  1: JUnit 4
  2: TestNG
  3: Spock
  4: JUnit Jupiter
Enter selection (default: JUnit Jupiter) [1..4] 1
```

```
Project name (default: gradleproject1): myapp
Source package (default: myapp): mypackage

> Task :init
Get more help with your project: https://docs.gradle.org/7.6.2/samples/sample_building_java_applications.html

BUILD SUCCESSFUL in 10m 10s
2 actionable tasks: 2 executed
E:\Eclipse_workplace\gradleproject1>
```

Thus I have successfully created the application named **myapp**.




We can edit the application program as follows -

**App.java**

```
* This Java source file was generated by the Gradle 'init' task.
*/
package mypackage;

public class App {
  public String getGreeting() {
    return "Welcome to first Gradle Application";
  }

  public static void main(String[] args) {
    System.out.println(new App().getGreeting());
  }
}
```

Now we can execute the above application using the gradle **tasks**. The Gradle Task is similar to **Maven Goals**. When we issue the command **gradle tasks** we can see the tasks that are available for the project -

```
Command Prompt

E:\Eclipse_workplace\gradleproject1>gradle tasks

> Task :tasks

------------------------------------------------------------
Tasks runnable from root project 'myapp'
------------------------------------------------------------

Application tasks
-----------------
run - Runs this project as a JVM application

Build tasks
-----------
assemble - Assembles the outputs of this project.
build - Assembles and tests this project.
buildDependents - Assembles and tests this project and all projects that depend on it.
buildNeeded - Assembles and tests this project and all projects it depends on.
classes - Assembles main classes.
clean - Deletes the build directory.
jar - Assembles a jar archive containing the main classes.
testClasses - Assembles test classes.

Build Setup tasks
-----------------
init - Initializes a new Gradle build.
wrapper - Generates Gradle wrapper files.

Distribution tasks
```

Now **to create a build of above application**, issue the command **gradle build**

```
Command Prompt

E:\Eclipse_workplace\gradleproject1>gradle build

BUILD SUCCESSFUL in 1m 2s
7 actionable tasks: 7 executed
E:\Eclipse_workplace\gradleproject1>
```

To execute the application issue the command **gradle run**

```
Command Prompt

E:\Eclipse_workplace\gradleproject1>gradle run

> Task :app:run
Welcome to first Gradle Application

BUILD SUCCESSFUL in 1s
2 actionable tasks: 1 executed, 1 up-to-date
E:\Eclipse_workplace\gradleproject1>
```

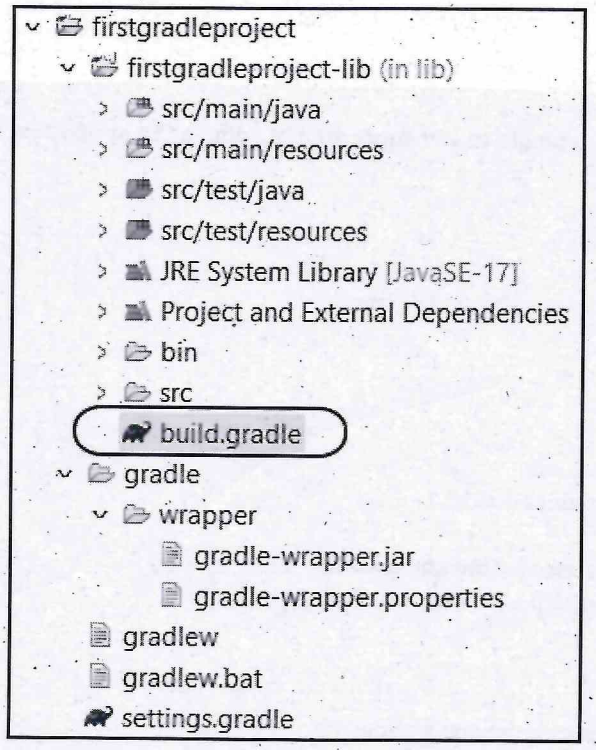
Thus a **build** directory gets created by above commands and **app.jar** gets generated. This proves that Gradle is a build automation tool which generates the jar or war files automatically.

To summarize, the gradle tasks are -

| | |
|---|---|
| **gradle tasks** | List the tasks that can be performed by gradle. |
| **gradle init** | Creates a gradle project |
| **gradle javadoc** | Generates Javadoc API documentation. |
| **gradle clean** | Deletes the build directory |
| **gradle build** | Compiles, unit tests and packages the application |
| **gradle run** | It is used to execute the main class |

## 2.16 Understand Build using Gradle

The directory structure in Gradle is as shown below -



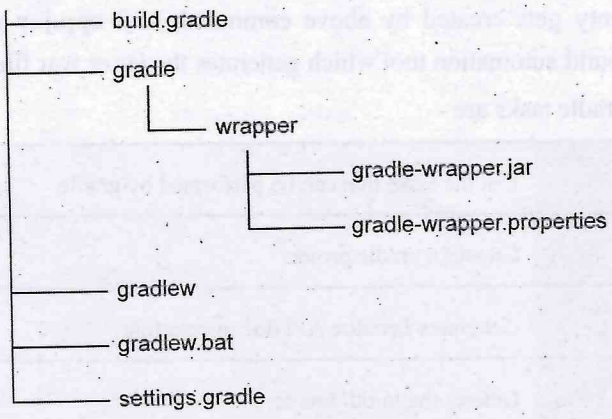



**Fig. 2.16.1 Gradle build structure**

**1) build.gradle -**

In **build.gradle** file we write our build script file. That means in this file we specify the dependencies, that are required by our project. All the build related aspects are specified in this file. In **Maven** we have **pom.xml** file to do the build related configurations, in the same way we have **build.gradle** file to do the build related configurations in **Gradle**. The sample **build.gradle** file is as follows -

**build.gradle**

```
plugins {
    // Apply the application plugin to add support for building a CLI application in Java.
    id 'application'
}

repositories {
    // Use Maven Central for resolving dependencies.
    mavenCentral()
}

dependencies {
    // Use JUnit test framework.
    testImplementation 'junit:junit:4.13.2'

    // This dependency is used by the application.
    implementation 'com.google.guava:guava:31.1-jre'
}

application {
    // Define the main class for the application.
    mainClass = 'mypackage.App'
}
```

Gradle provides a **Domain Specific Language (DSL)**, for describing builds. This uses the **Groovy** language. All the tasks and plugins are defined in this file.

When we run a gradle command, it looks for a file called **build.gradle** in the current directory. It is a build configuration script. The build script defines a project and its tasks.

The **build.gradle** file contains three default sections. They are as follows :

- **plugins :** In this section, we can apply the java-library plugin to add support for java library.
- **Repositories :** In this section, we can declare internal and external repository for resolving dependencies. We can declare the different types of repository supported by Gradle like Maven, Ant, and Ivy.
- **Dependencies :** In this section, we can declare dependencies that are necessary for a particular subject.

**2) gradle Directory -**

Another important aspect of gradle build mechanism is a **gradle** directory.

Inside this **gradle** directory there is one more directory called **wrapper**. Gradle Wrapper, also called **Wrapper** in short.

The **wrapper** contains some important files as describe below -

- **gradle-wrapper.jar** contains code for downloading the Gradle distribution specified in the gradle-wrapper.properties file
- **gradle-wrapper.properties** contains Wrapper runtime properties - most importantly, the version of the Gradle distribution that is compatible with the current project
- **gradlew** is the script that executes Gradle tasks with the Wrapper on Linux and MacOS machines.
- **gradlew.bat** is the gradlew equivalent batch script for Windows machines

The main benefits of Wrapper are that we can build a project with Wrapper on any machine without the need to install Gradle first. That means if on one machine if we create a **gradle project** and if we run that project on another machine then we need not have to install **Graddle** software on that machine. This is due to the **graddle-wrapper** file.

**3) settings.gradle -**

The settings.gradle file is used to configure the **global settings** of gradle project.

**Review Questions**

1. *Explain in detail the Gradle build structure.*
2. *Explain build.gradle in detail.*

## 2.17 Maven Vs. Gradle

| Sr. No. | Maven | Gradle |
|---|---|---|
| 1. | Maven is a tool used form generating **Java based** projects. | Gradle is a tool which can be used to develop **domain-specific language** projects |
| 2. | It is uses the **XML(pom.xml)** to create project structure. | It uses Groovy based **Domain Specification Language(DSL)** for creating the project structure. |
| 3. | The purpose of the Maven tool-based project is to **finish the project** within the given timeline. | The purpose set for Gradle based tool project is to **incorporate new functionality** into the project. |
| 4. | The **Java compilation is compulsory** for the Maven tool. | The **java compilation is not compulsory** for the Gradle tool. |
| 5. | It does not create local temporary files during software creation, and is hence – **slower**. | It **performs better** than maven as it optimized for tracking only current running task. |
| 6. | It supports software development in Java, Scala, C#, and Ruby. | It supports software development in Java, C, C++, and Groovy. |

### Review Question

1. *Give the difference between Maven and Gradle.*

## 2.18 Two Marks Questions with Answers

**Q.1 What is Maven ?**

**Ans. :** Maven is a project management tool. This tool is used for build, automation and project management tool is used for managing Java projects.

**Q.2 What problem will occur if we build the Java project without Maven ?**

**Ans. : If we choose not to use Maven for building the Java projects then we may encounter following problem -**

1) **Dependency management :** Without Maven we need to manage all the project dependencies manually. It includes the complex tasks such as downloading library files, ensuring their compatibility with the undergoing project, managing the version control. It can be time-consuming and error-prone.

2) **Lack of plugins and community support :** Maven has support for plugins that can simplify various tasks such as deployment and documentation generation. Without Maven, lack of Plugins forces the developer to perform deployment and documentation tasks manually. Which can be time-consuming and error-prone.

**Q.3 What are the features of Maven ?**

**Ans. :**

1) Maven provides simple project setup that follows best practices.
2) Maven provides superior dependency management, automatic updating and dependency closures.
3) It is extensible, with the ability to easily write plugins in Java or scripting languages.

**Q.4 What id groupId and artifactId in Maven ?**

**Ans. : groupId :** It is sub element of project. It specifies the id for the project group.

**artifactId :** It is a sub element of project. This is generally refers to the name of the project. The artifact ID is also used as part of the name of the JAR, WAR or EAR file produced when building the project.

**Q.5 What is snapshot artifact ?**

**Ans. :** A snapshot artifact indicates that the project is under development. When we install or publish a component, Maven checks the version attribute. If it contains the string "SNAPSHOT", Maven converts this key into a date and time value in UTC (Co-ordinated Universal Time) format.

**Q.6 What is the significance of POM.XML ?**

**Ans. :** POM stands for Project Object Model. It contains information related to the project and configuration information such as

- dependencies,
- plugins,
- source directory,
- goals and so on

Maven reads pom.xml file to accomplish its configuration and operations.

**Q.7 Enlist the phases in Maven build lifecycle.**

**Ans. :**

1) Validate 2) Compile 3) Test Compile
4) Test 5) Package 6) Install 7) Deploy

**Q.8 What is the purpose of Maven Goal ?**

**Ans. :** The purpose of each goal in Maven is to perform in specific task. When we build our Maven project, we need to specify the Goal. Some of the popular goals are -

- Validate
- compile
- test Compile
- package
- install
- deploy

**Q.9 What is the purpose of Maven clean ?**

**Ans. :** The purpose of clean phase is to clean up the project and make it ready for fresh compile and deployment. It removes all files generated by the previous build.

**Q.10 What will be the result of specifying the package as a Maven goal ?**

**Ans. :** If we specify the Maven goal as "package" then the code will be compiled, tested and then packaged into some distributable format such as - jar or war.

**Q.11 Explain in brief the concept of Maven profiles.**

**Ans. :**

- Maven Profile is an important feature of Maven. Using Profile we can customize the build for different environments.
- Profiles are portable for different build environments. The build environment means a specific environment for production, testing or development environment instances. So in order to configure these instances Maven provides the various features for the build using profile.

**Q.12 What is Maven repository ?**

**Ans. :** Maven repository is a directory where all the project jars, plugins, library jars or any other project-related artifacts are stored and these can be accessed by Maven easily. Maven Central can be accessed from **https://repo.maven.apache.org/maven2/**.

**Q.13 Enlist any two limitations of Maven.**

**Ans. :** The following are some disadvantages or limitations of Maven -

1) Installation of Maven in each working system is required for executing the Maven-based project.
2) We can not implement the dependency using Maven if the Maven code for existing dependency is not available.

**Q.14 What is Gradle ?**

**Ans. :** Gradle is an Open source build automation tool. The build automation tool is a tool that automates the creation of software builds. Build automation is the process of automating the retrieval of source code, compiling it into binary code, executing automated tests and publishing it into a shared, centralized repository.

**Q.15 Enlist the features of Gradle.**

**Ans. : 1) Free and Open Source :** It is an open source tool.

2) **High Performance :** Gradle finishes the tasks efficiently by considering the outputs from the previous execution.

3) **Better Support :** Gradle is popular for providing the support. The Gradle can provide support to build ANT projects. Tasks can be imported from ANT build projects and can be used in Gradle project.

4) **Scaling :** Gradle can increase the productivity, from simple and single project to huge multi-project builds.

**Q.16 Explain the Build-gradle used in Gradle.**

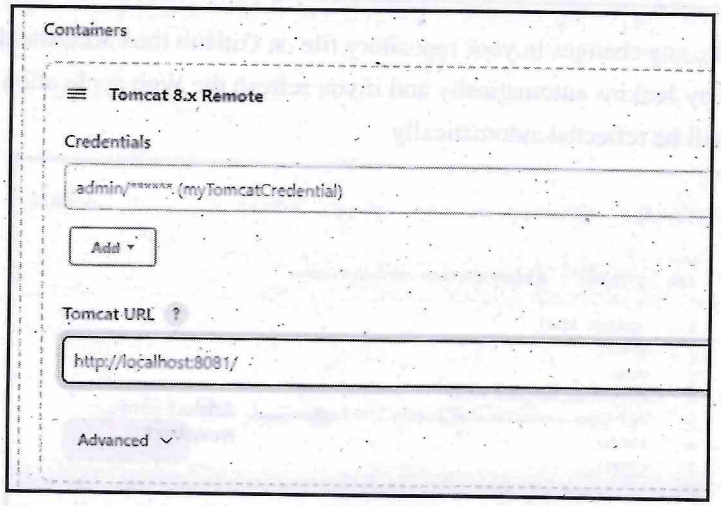
**Ans. :** Gradle builds the build script for - **1. project 2. task**. Every Gradle build represents one or more projects. The build script is written using the domain specification language called groovy. This script is saved as **build.gradle**. That means in this file we specify the dependencies, that are required by our project. All the build-related aspects are specified in this file.

**Q.17 Differentiate between Maven and Gradle.**

**Ans. :** Refer section 2.17.

❑❑❑

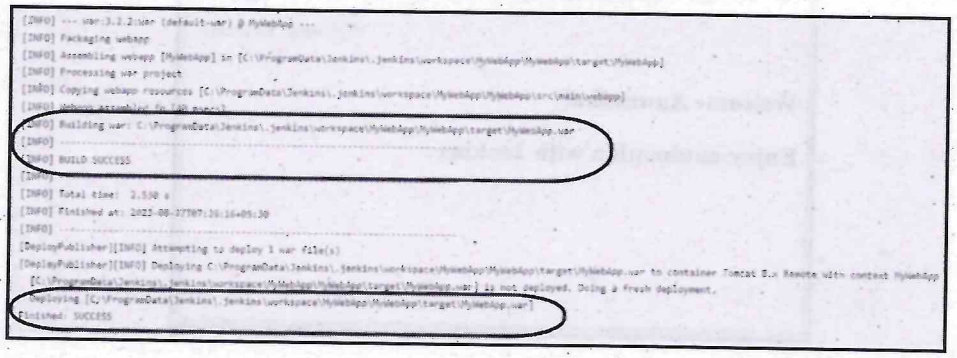
**Notes**



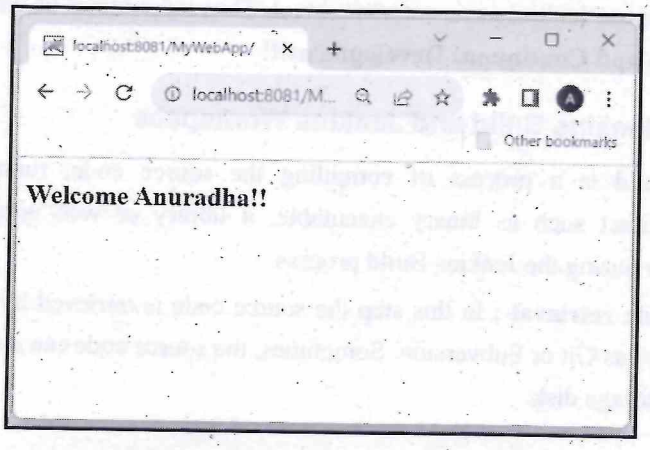

Finally Click Apply and Save.

**Step 9 :** Start your local Tomcat server. Then on Jenkins page, click the **Build Now** for building the automation.

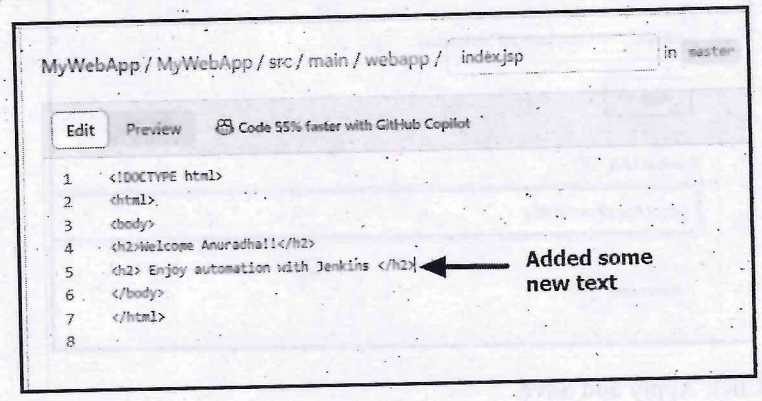
On Successful Build we get following output on Console.



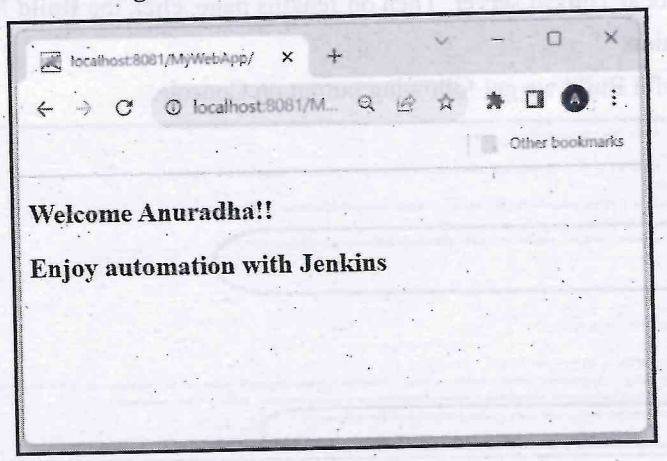

**Step 10 :** Open the web browser and enter the URL of your Web Application.



**Step 11 :** If you make any changes in your repository file on GitHub then accordingly Build will be generated by Jenkins automatically and if you refresh the Web application URL, those changes will be reflected automatically.




The **commit the changes** GitHub. Now the output will be -




Thus when developers edit the code, those changes will be automatically reflected as an output, if we use Jenkins as an automated tool. Thus it facilitates us with **Continuous Integration and Continuous Development!!!**

## 3.10 Creating a Jenkins Build and Jenkins Workspace

- In Jenkins Build is a process of compiling the source code, running tests, creating deployable artifact such as binary executable, a library or web application. Following activities occur during the Jenkins Build process.

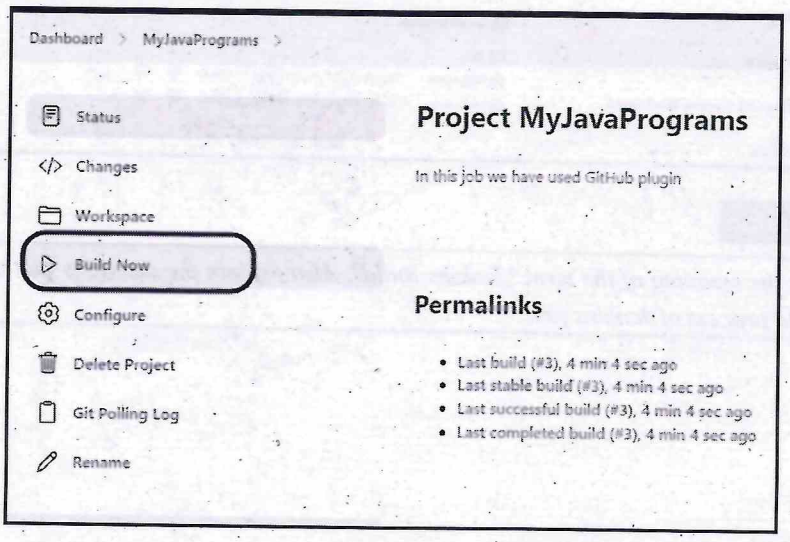
1. **Source code retrieval :** In this step the source code is retrieved by the version control system such as Git or Subversion. Sometimes, the source code can also be retrieved from the local storage disk.

2. **Workspace creation :** Jenkins creates a working directory which is known as workspace where it stores all the files necessary for the build process.
3. **Build environment configuration :** Jenkins allows to specify the files, libraries and dependencies required for the build, which is called configuration process.
4. **Build process :** Following activities are carried out during the actual build process -
   a. **Compilation :** Source code is compiled into executable binaries or libraries.
   b. **Testing :** Running test cases to check the correctness of the source code.
   c. **Artifact Generation :** The artifacts such as jar or war files are generated.
5. **Reporting :** Jenkins collect and generate reports during the build process. For instance the HTML report can be published by Jenkins.
6. **Artifact archiving :** Jenkins collect the artifact that are ready for deployment.
7. **Deployment :** The artifacts that are deployed in the executable environment. For example if the war file is generated during the Jenkins Build can be deployed on the Tomcat container for execution.
8. **Logging :** Jenkins maintains detailed logs of each build, including console output and build artifact.

### ➢ Steps to Create Jenkins Build

**Step 1 :** Open the web browser and access the web interface of Jenkins by providing the username and password.

**Step 2 :** Click on "New Item" on the Jenkins dashboard to create a new job or select the already created job. Once job is ready, click on **Build Now.**



**Step 3 :** If there exists some errors, then Jenkins will display the errors on Console output otherwise the actual output can be seen on the console.

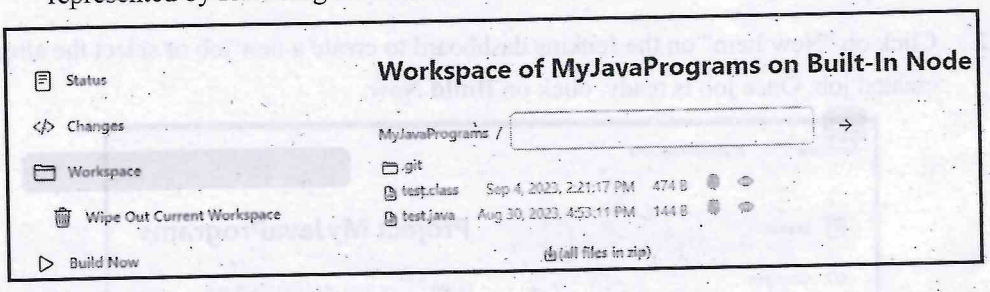
For example - Following is a sample output of Jenkins job that can be collected from Console window -

```
Started by user Anuradha P
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\.jenkins\workspace\MyJavaPrograms
The recommended git tool is: NONE
No credentials specified
 > C:\Program Files\Git\bin\git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\.jenkins\workspace\MyJavaPrograms\.git # timeout=10
Fetching changes from the remote Git repository
 > C:\Program Files\Git\bin\git.exe config remote.origin.url https://github.com/AnuradhaP/MyJavaPrograms.git # timeout=10
Fetching upstream changes from https://github.com/AnuradhaP/MyJavaPrograms.git
 > C:\Program Files\Git\bin\git.exe --version # timeout=10
 > git --version # 'git version 2.41.0.windows.3'
 > C:\Program Files\Git\bin\git.exe fetch --tags --force --progress -- https://github.com/AnuradhaP/MyJavaPrograms.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
 > C:\Program Files\Git\bin\git.exe rev-parse "refs/remotes/origin/master^{commit}" # timeout=10
Checking out Revision ae9e4f19235b9940812405cbf0c73092735d326f (refs/remotes/origin/master)
 > C:\Program Files\Git\bin\git.exe config core.sparsecheckout # timeout=10
 > C:\Program Files\Git\bin\git.exe checkout -f ae9e4f19235b9940812405cbf0c73092735d326f # timeout=10
Commit message: "Changes in the test.java"
 > C:\Program Files\Git\bin\git.exe rev-list --no-walk ae9e4f19235b9940812405cbf0c73092735d326f # timeout=10
[MyJavaPrograms] $ cmd /c call C:\WINDOWS\TEMP\jenkins7497915670774259367.bat

C:\ProgramData\Jenkins\.jenkins\workspace\MyJavaPrograms>javac test.java

C:\ProgramData\Jenkins\.jenkins\workspace\MyJavaPrograms>java test.java
Good Morning Path!!!
Good Morning Path!!!
```

The workspace for the Jenkins job gets created automatically. Sample workspace is represented by following screenshot.




## Review Question

1. *Explain the meaning of the term "Jenkins Build". Also explain the activities that occure during the Build process of Jenkins job.*

## 3.11 Two Marks Questions with Answers

**Q.1 What is Jenkins ?**

**Ans. :**

- Jenkins is a open source automation tool which allows Contiguous Integration (CI). It is written in Java.
- Jenkins build and test our software projects continuously which becomes easy for developers to integrate the changes in the project.

**Q.2 Why Jenkins is so popular in DevOps ?**

**Ans. :**

- Jenkins is a popular tool in the DevOps community because it allows to automate the software development process, thereby reducing the cost of manual testing and increasing quality.
- Jenkins supports large number of plugins which makes it easy to configure and customize any project.

**Q.3 Explain the term continuous integration.**

**Ans. :** In continuous integration whenever the code commit occurs a build must be triggered. The job of triggering the build for every commit can be automated by Jenkins.

**Q.4 Explain the use of freestyle project in Jenkins.**

**Ans. :** Jenkins freestyle projects allow users to automate simple jobs, such as running tests, creating and packaging applications, producing reports or executing commands. Freestyle projects are repeatable and contain both build steps and post-build actions.

**Q.5 Enlist the benefits of using plugins in Jenkins.**

**Ans. :** Following are some benefits that are observed when the Jenkins plugins are used -

1. **Extended functionality :** Plugins can be used to add new features to Jenkins. Many plugins are open-source and maintained by the Jenkins community. This means you can benefit from the contributions and improvements made by a large user base.
2. **Task automation :** Plugins can be used to automate various tasks related to software development such as building, testing, deployment of library files to executable environment.

**Q.6 Enlist some plugins used in Jenkins.**

**Ans. :**

1) Git plugin 2) Docker plugin 3) Amazon EC2 plugin
4) SonarQube plugin 5) Jira plugin.

**Q.7 What is the use of HTML publisher plugin ?**

**Ans. :** The HTML publisher plugin is used to generate and publish the HTML-based reports as a part of build and deployment stage.

**Q.8 Enlist the parameter types of extended choice parameter plugin.**

**Ans. :**

1. **Single select :** Allows users to choose a single option from a list.
2. **Multi-select :** Allows users to select multiple options from a list.
3. **Checkboxes :** Presents options as checkboxes for multiple selections.
4. **Radio buttons :** Presents options as radio buttons for single selection.
5. **PTT (Progressive Trace Text) :** Provides a text box with autocomplete suggestions based on the user's input.

❑❑❑